

LexiConnect – Secure Legal Service Discovery and Case-Based Appointment Management Platform

D. Thenujayan UOG0422019

Dinuga Vithana UOG0422015

Purindada Udavi UOG0422032

Methsarani Kavindi UOG0422003

Yasasthi Chapa UOG0423083

Name of the supervisor:

Mr. Janith Udayanga

February 2026

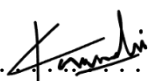
Declaration

We certify that this dissertation does not incorporate, without acknowledgement, any material previously submitted for a degree or diploma in any University or Institute and, to the best of our knowledge and belief, it does not contain any material previously published or written by another person except where due reference is made in the text.

Signature of Candidate: 

Date: 01/02/2026

Name of Candidate: Devmith Thenujayan

Signature of Candidate: 

Date: 01/02/2026

Name of Candidate: Methsarani Kavindi Wanasingha

Signature of Candidate: 

Date: 01/02/2026

Name of Candidate: Dinuga Lavanya Vithana

Signature of Candidate: 

Date: 01/02/2026

Name of Candidate: Purindada Udavi

Signature of Candidate:

Date: 01/02/2026

Name of Candidate: Yasasthi Chapa

Countersigned by:

Signature of Supervisor:

Date: 01/02/2026

Name of Supervisor: **Mr. Janith Udayanga**

Abstract

LexiConnect is a secure, web-based legal service discovery and case-based appointment management platform developed by our team to address structural inefficiencies in accessing legal services. In traditional legal systems, clients without personal legal connections struggle to find suitable lawyers, newly qualified lawyers face difficulty gaining exposure, and apprentice lawyers lack structured digital support during their mandatory training period under practicing lawyers. Additionally, manual appointment scheduling and informal case handling often result in conflicts, poor traceability, and limited accountability.

To address these challenges, LexiConnect enables clients to post legal issues with open comment discussions, allowing lawyers to review, request, and manage cases based on specialization and availability. Newly registered and less popular lawyers are provided equal visibility through a centralized case feed, while apprentice lawyers can be formally associated with cases under supervising lawyers, supporting real-world legal training requirements. Appointment booking is tightly coupled with lawyer-defined availability and service packages, preventing scheduling conflicts and ensuring realistic workflows.

The system implements a robust Role-Based Access Control (RBAC) mechanism where privileges, rather than fixed roles, govern system actions. This ensures secure, auditable, and flexible authorization across all modules. LexiConnect is implemented using React and FastAPI, with PostgreSQL as the backend database, and follows an Agile incremental development model. Comprehensive testing and evaluation confirm that the system satisfies both functional and non-functional requirements while remaining scalable and legally respectful.

Acknowledgements

We would like to express our sincere gratitude to our supervisor, **Mr. Janith Udayanga**, for his continuous guidance, valuable feedback, and encouragement throughout the development of this project. We also thank the academic staff of the Software Engineering program for providing the knowledge, facilities, and support required to successfully complete this work. Finally, we acknowledge the collaborative effort of all group members — **D. T. Thenujayan, Dinuga Vithana, Purindada Udavi, Methsarani Kavindi, and Yasasthi Chapa** — whose combined contributions across analysis, design, implementation, and testing made this project possible.

Table of Contents

Chapter 1 – Introduction	10
1.1 Background and Motivation	10
1.2 Problem Definition	10
1.3 Aims and Objectives	10
1.4 Scope of the Project	11
1.5 Novelty of the Work	11
1.6 Overview of the Dissertation Structure	12
Chapter 2 – Analysis	13
2.1 Existing System Analysis Using Flow Charts	13
2.2 Review of Similar Systems	13
2.3 Functional and Non-Functional Requirements	14
2.4 Process Model Selection and Justification	15
Chapter 3 – Design	16
3.1 Relevant Design Diagrams	16
3.2 User Interface Design with Wireframes	23
Chapter 4 – Implementation	27
4.1 Implementation Environment	27
4.2 Overall System Implementation Structure	28
4.3 Authentication and Authorization Implementation	29
4.4 Case Management Implementation	31
4.5 Apprentice Lawyer Workflow Implementation	31
4.6 Availability and Booking Implementation	33
4.7 Admin Access Control Implementation	34
4.8 Audit Logging Implementation	35
4.9 Frontend Implementation	35
4.10 Code Conventions and Standards	36
Chapter 5 – Evaluation	37
5.1 Testing Strategy and Test Plan	37

5.2 Unit Testing.....	38
5.3 Integration Testing.....	39
5.4 System Testing.....	39
5.5 Role-Based Access Control (RBAC) Testing.....	40
5.6 Apprentice Lawyer Workflow Testing.....	40
5.7 Error Handling and Edge Case Testing.....	41
5.8 User Acceptance Testing (UAT).....	42
5.9 Test Evidence and Screenshots.....	42
5.10 Evaluation Summary.....	43
Chapter 6 – Conclusion.....	44
6.1 Achievement of Project Objectives.....	44
6.2 Evaluation of Results.....	45
6.3 Lessons Learned.....	45
6.4 Limitations of the System.....	45
6.5 Future Enhancements.....	46
6.6 Final Remarks.....	47

List of Figures (Final – LexiConnect)

Chapter 2 – Analysis

- Figure 2.1 – Existing Manual Legal Service Workflow
- Figure 2.2 – Agile Incremental Development Model

Chapter 3 – Design

- Figure 3.1 – System Architecture Diagram (LexiConnect)
- Figure 3.2 – Backend Module Interaction Diagram
- Figure 3.3 – Entity Relationship (ER) Diagram
- Figure 3.4 – Role-Based Access Control (RBAC) Design Diagram
- Figure 3.5 – Apprentice Lawyer Workflow Diagram
- Figure 3.6 – Client Dashboard Wireframe
- Figure 3.7 – Case Posting Interface Wireframe
- Figure 3.8 – Lawyer Dashboard Wireframe
- Figure 3.9 – Incoming Bookings Interface Wireframe
- Figure 3.10 – Apprentice Case View Wireframe
- Figure 3.11 – Admin Access Control Panel Wireframe

Chapter 4 – Implementation

- Figure 4.1 – High-Level Module Interaction Diagram
- Figure 4.2 – RBAC Enforcement Flow
- Figure 4.3 – Apprentice Case Assignment Flow
- Figure 4.4 – Booking Workflow Diagram

Chapter 5 – Evaluation

- Figure 5.1 – Testing Strategy Overview Diagram

List of Tables

Chapter 2 – Analysis

- **Table 2.1** – Comparison of Similar Systems
- **Table 2.2** – Functional Requirements
- **Table 2.3** – Non-Functional Requirements

Chapter 5 – Evaluation

- **Table 5.1** – Sample Unit Test Cases
- **Table 5.2** – Integration Test Cases
- **Table 5.3** – RBAC Test Cases
- **Table 5.4** – Apprentice Workflow Test Cases
- **Table 5.5** – Edge Case Test Scenarios
- **Table 5.6** – User Acceptance Testing (UAT) Summary

Appendices

- **Table D.1** – Booking Summary Report
- **Table D.2** – Case Activity Report
- **Table D.3** – User Growth Report

List of Acronyms

Acronym	Meaning
RBAC	Role-Based Access Control
JWT	JSON Web Token
API	Application Programming Interface
ORM	Object Relational Mapping
UI	User Interface
UAT	User Acceptance Testing
ER	Entity Relationship

Chapter 1 – Introduction

1.1 Background and Motivation

Access to legal services is a critical requirement in any society; however, in many real-world contexts, particularly in Sri Lanka, legal service discovery and appointment handling remain largely informal and inefficient. Clients often rely on personal referrals or physical visits to identify lawyers, which disadvantages individuals without established legal networks. At the same time, newly qualified lawyers and apprentice lawyers face significant barriers in gaining practical exposure and clients, as legal work is often reputation-driven.

Furthermore, traditional legal workflows rely heavily on manual scheduling, phone calls, and paper-based documentation, leading to appointment conflicts, lack of transparency, and poor traceability of case-related interactions. These issues motivated our team to design and implement **LexiConnect**, a digital platform that modernizes legal service access while remaining legally respectful and academically appropriate.

1.2 Problem Definition

The key problems identified are:

- Difficulty for clients to find suitable lawyers for specific legal issues
- Limited visibility and opportunities for newly qualified lawyers
- Lack of structured digital support for apprentice lawyers undergoing mandatory training
- Manual and error-prone appointment scheduling
- Absence of fine-grained access control and auditability in existing systems

1.3 Aims and Objectives

The primary aim of this project is to design and develop a secure and scalable legal service management platform. The specific objectives are:

1. To provide a centralized platform for legal service discovery
2. To enable structured case posting with open comment discussions
3. To support apprentice lawyers working under supervising lawyers

4. To facilitate case-based appointment booking with availability constraints
5. To implement fine-grained role-based access control using privileges
6. To improve visibility and fairness for newly qualified lawyers
7. To ensure system security, auditability, and maintainability

1.4 Scope of the Project

In-Scope Functionalities

- User registration and authentication
- Case creation and open comments
- Lawyer case feed and case requests
- Apprentice lawyer assignment to cases
- Lawyer availability and appointment booking
- Document upload and case continuity
- Admin access control and audit logs

Out-of-Scope Functionalities

- Online payments
- AI-based legal advice
- Real-time chat
- Mobile application development

1.5 Novelty of the Work

Unlike generic lawyer directories or appointment systems, LexiConnect integrates **case-based workflows**, **apprentice lawyer participation**, and **privilege-driven access control** into a single platform. The use of RBAC at both backend and frontend levels ensures strong security while maintaining flexibility for administrative control.

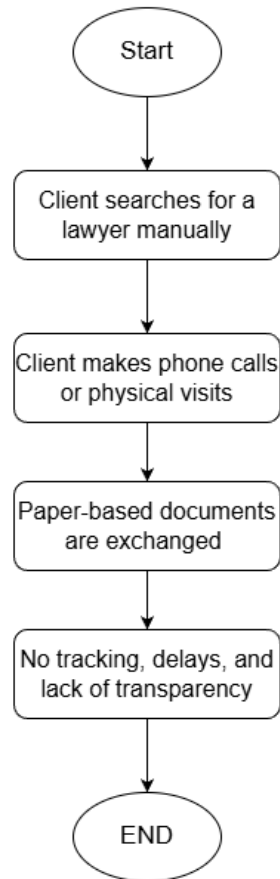
1.6 Overview of the Dissertation Structure

- **Chapter 2** presents system analysis and requirements
- **Chapter 3** discusses system design and UI wireframes
- **Chapter 4** explains implementation details
- **Chapter 5** evaluates the system through testing
- **Chapter 6** concludes the dissertation and outlines future work

Chapter 2 – Analysis

2.1 Existing System Analysis Using Flow Charts

In the traditional legal service workflow, clients typically identify lawyers through personal referrals or physical visits to legal chambers. Appointment scheduling is handled manually, often via phone calls, resulting in limited availability visibility and frequent scheduling conflicts.



This workflow lacks transparency, structured case tracking, and digital record keeping, making it inefficient for both clients and legal professionals.

2.2 Review of Similar Systems

System 1: Online Lawyer Directories

Many existing platforms provide basic lawyer listings and contact details. However, these systems do not support structured case posting, apprentice involvement, or appointment continuity. Visibility is often biased toward popular lawyers.

System 2: Generic Appointment Scheduling Systems

While scheduling tools exist, they are not tailored for legal workflows and lack features such as case association, privilege-based access, and legal auditability.

These limitations informed the design decisions adopted in LexiConnect.

2.3 Functional and Non-Functional Requirements

Functional Requirements

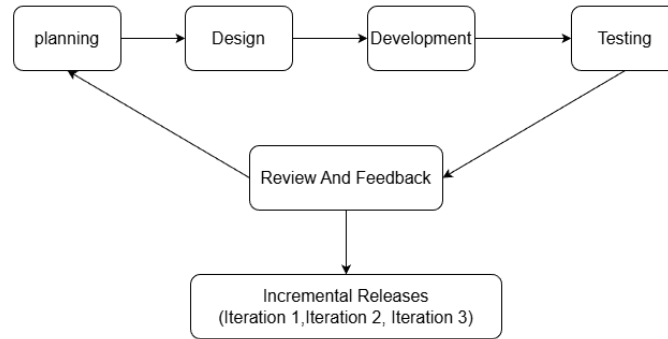
ID	Requirement
FR1	The system shall allow users to register and authenticate securely
FR2	Clients shall post legal cases with open comments
FR3	Lawyers shall view and request cases
FR4	Apprentice lawyers shall be assigned under supervising lawyers
FR5	Lawyers shall define availability schedules
FR6	Clients shall book appointments based on availability
FR7	Admins shall manage roles and privileges

Non-Functional Requirements

- **Security:** JWT authentication and RBAC enforcement
- **Performance:** Efficient API responses
- **Reliability:** Consistent booking and case handling
- **Auditability:** Complete action logging

2.4 Process Model Selection and Justification

An **Agile Incremental Development Model** was selected due to evolving requirements, continuous supervisor feedback, and the need for iterative validation. Features were developed and integrated incrementally, allowing early detection of issues and continuous improvement.



Chapter 3 – Design

This chapter describes the design of the LexiConnect system. A methodical and structured design approach was followed to ensure that the system architecture, database structure, access control model, and user interfaces collectively satisfy the functional and non-functional requirements identified in Chapter 2. Design decisions were evaluated against alternative strategies, and the most suitable approaches were selected with justification.

3.1 Relevant Design Diagrams

3.1.1 Design Strategy and Alternatives

During the design phase, several alternative strategies were evaluated:

1. Development Approach

- Developing the system fully from scratch
- Using third-party legal service platforms or plugins

The first approach was selected to maintain full control over security, workflows, and academic compliance.

2. Deployment Model

- Standalone desktop application
- Client–server web-based architecture

A **client–server web architecture** was selected to allow accessibility, scalability, and centralized data management.

3. Technology Stack

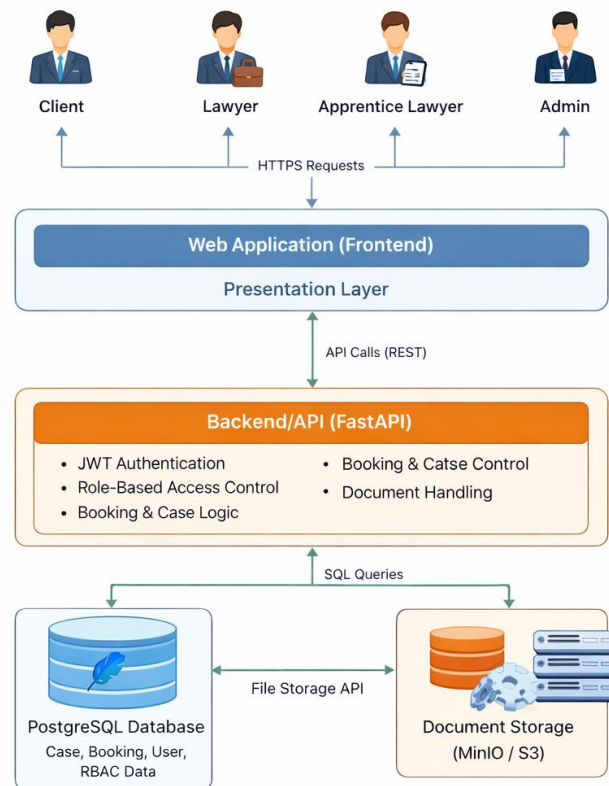
- PHP-based frameworks
- Java Spring Boot
- Python FastAPI

FastAPI was selected due to its performance, clean dependency injection model, and suitability for privilege-based authorization.

3.1.2 System Architecture Design

LexiConnect follows a **three-tier architecture** consisting of:

- Presentation Layer (Frontend)
- Application Layer (Backend API)
- Data Layer (Database)



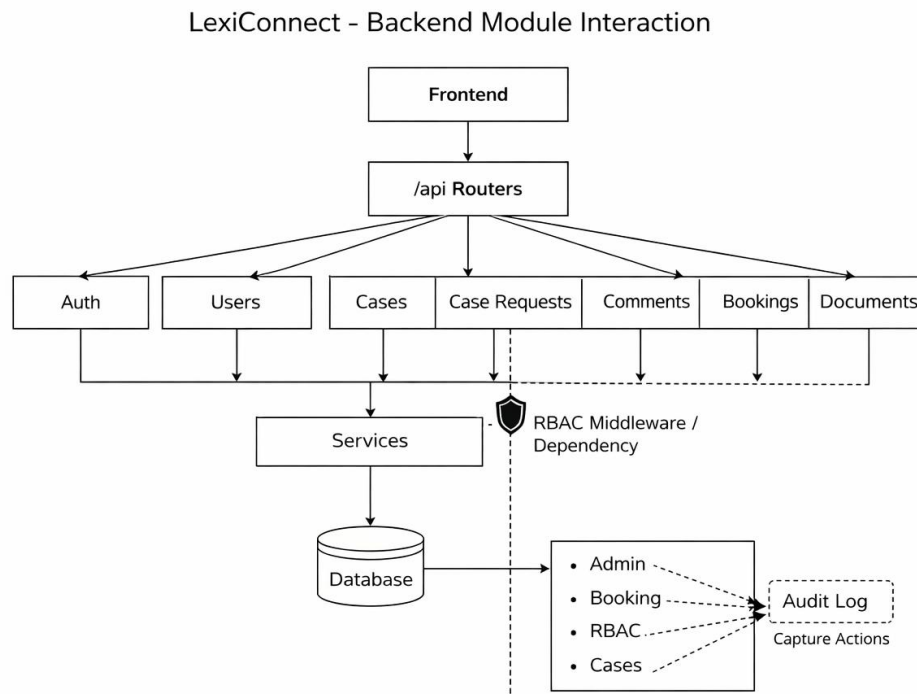
Description:

The frontend is implemented using React and communicates with the backend via RESTful APIs. The backend, developed using FastAPI, handles authentication, authorization, business logic, and data persistence. PostgreSQL is used as the relational database. Role-Based Access Control (RBAC) is enforced at the backend using privilege checks and mirrored at the frontend for UI-level access control.

3.1.3 Module-Level Design

The backend system is divided into feature-based modules to ensure maintainability and ownership clarity:

- Authentication Module
- User Management Module
- Case Management Module
- Booking & Availability Module
- Apprentice Lawyer Module
- RBAC Module
- Audit Logging Module



Each module exposes clearly defined APIs and interacts with other modules through service layers, avoiding tight coupling.

3.1.4 Database Design (ER Diagram)

The database was designed using a relational model to maintain consistency and enforce data integrity.

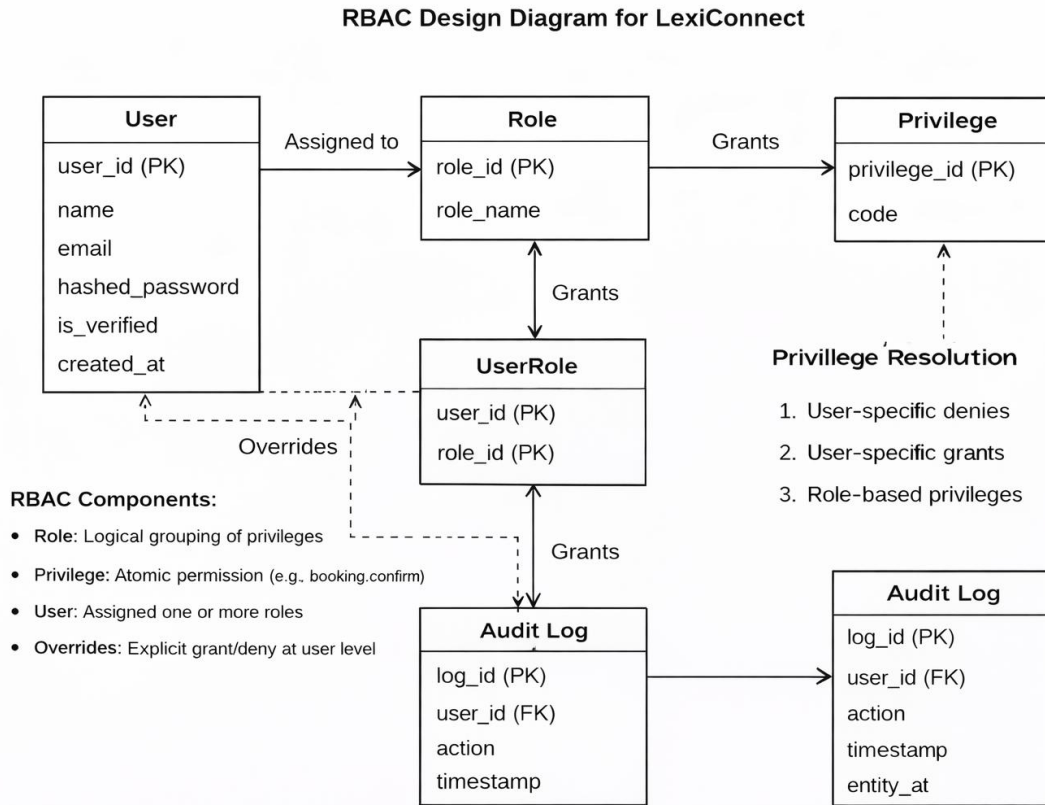
Key entities include:

- Users
- Roles
- Privileges
- Role–Privilege Mapping
- User–Privilege Overrides
- Cases
- Case Requests
- Case Comments
- Apprentice Case Assignments
- Bookings
- Availability Slots
- Audit Logs

- Normalized schema to avoid redundancy
- Foreign key constraints for data integrity
- Audit logs linked to users for traceability

3.1.5 Role-Based Access Control (RBAC) Design

LexiConnect implements a **privilege-centric RBAC model**, rather than a rigid role-based model.



RBAC Components:

- **Role:** Logical grouping of privileges
- **Privilege:** Atomic permission (e.g., booking.confirm)
- **User:** Assigned one or more roles
- **Overrides:** Explicit grant/deny at user level

Privilege Resolution Order:

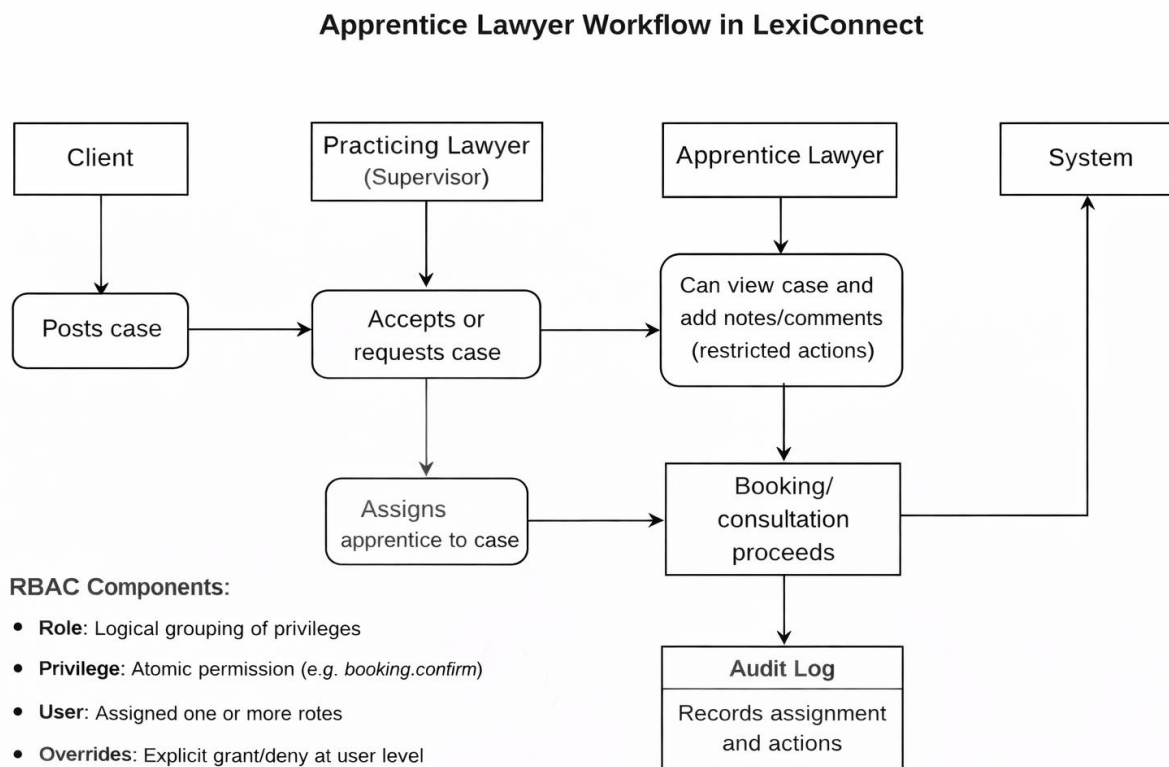
1. User-specific denies
2. User-specific grants
3. Role-based privileges

This design allows administrators to:

- Disable actions (e.g., booking confirmation) without changing roles
- Temporarily restrict users
- Maintain full auditability

3.1.6 Apprentice Lawyer Design

In Sri Lanka, newly graduated lawyers are required to undergo a training period under a practicing lawyer. LexiConnect explicitly supports this requirement.



Design Features:

- Apprentice lawyers are linked to supervising lawyers
- Apprentices can view assigned cases
- Apprentices can add notes and comments

- Critical actions remain restricted via RBAC

This ensures legal correctness while providing structured digital exposure.

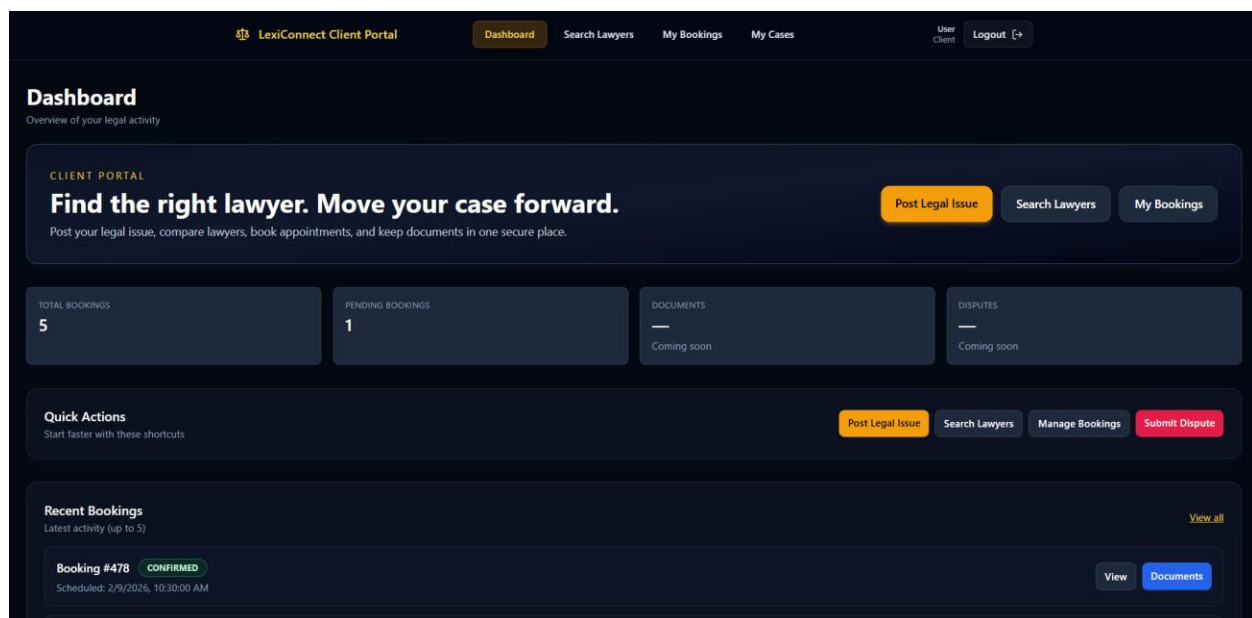
3.2 User Interface Design with Wireframes

3.2.1 UI Design Principles

The following principles guided UI design:

- Professional and minimal appearance
- Clear separation of roles
- Privilege-aware rendering
- Avoidance of clutter
- Legal-grade clarity

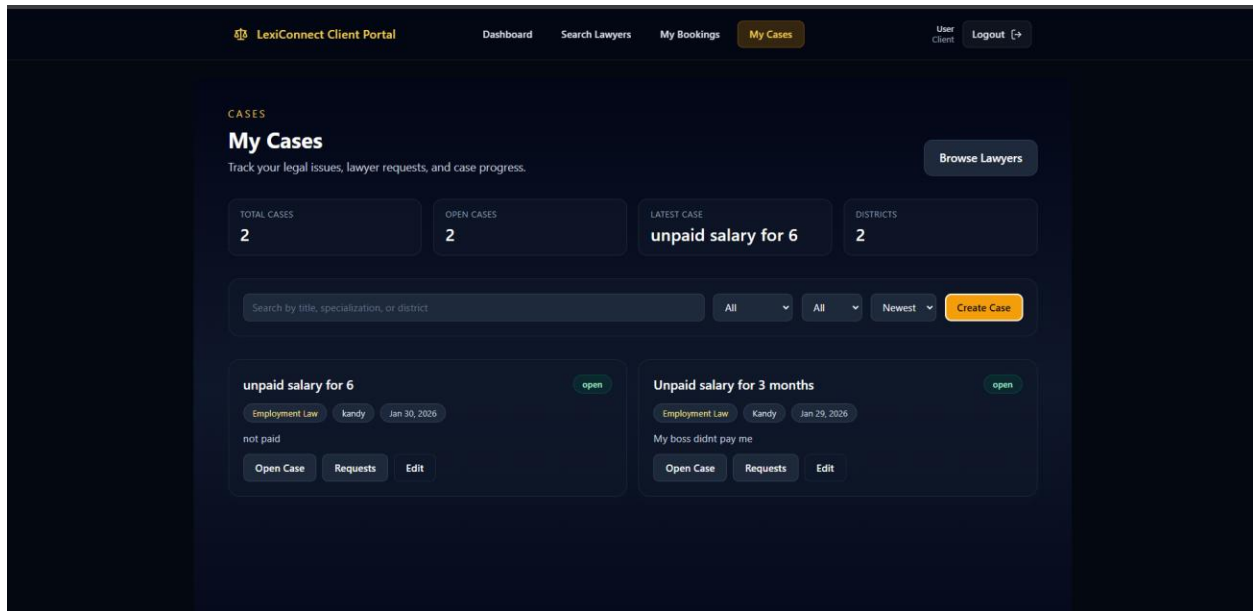
3.2.2 Client Interfaces



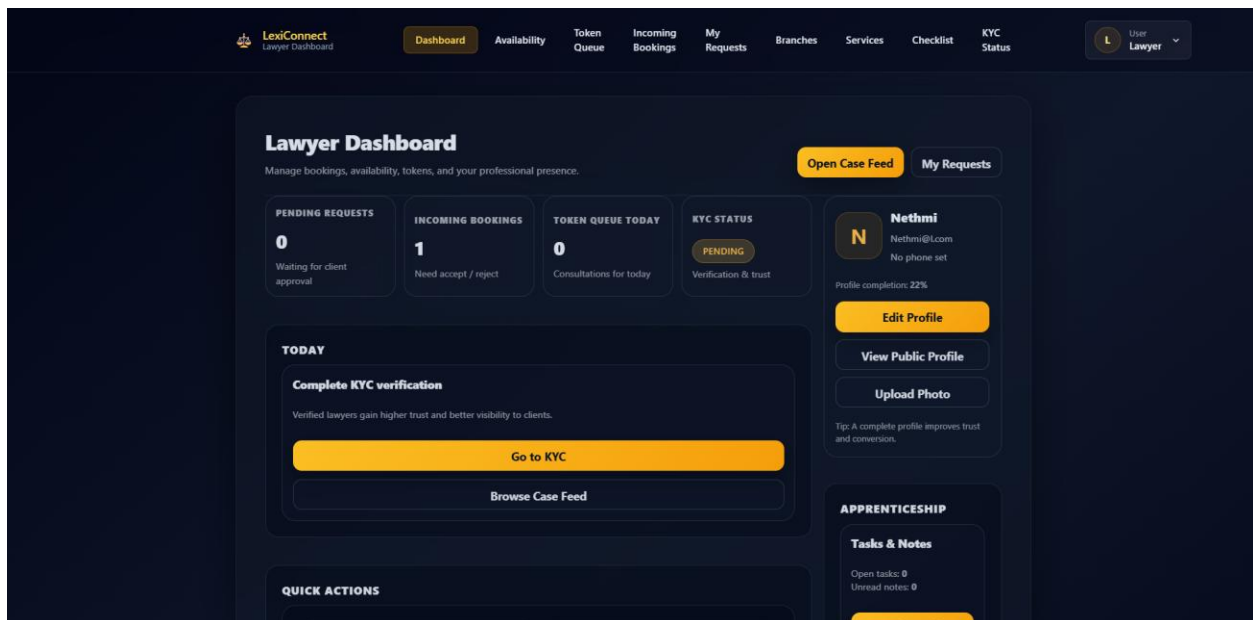
Key Features:

- Case creation
- Open comment discussions
- Booking history

- Document uploads



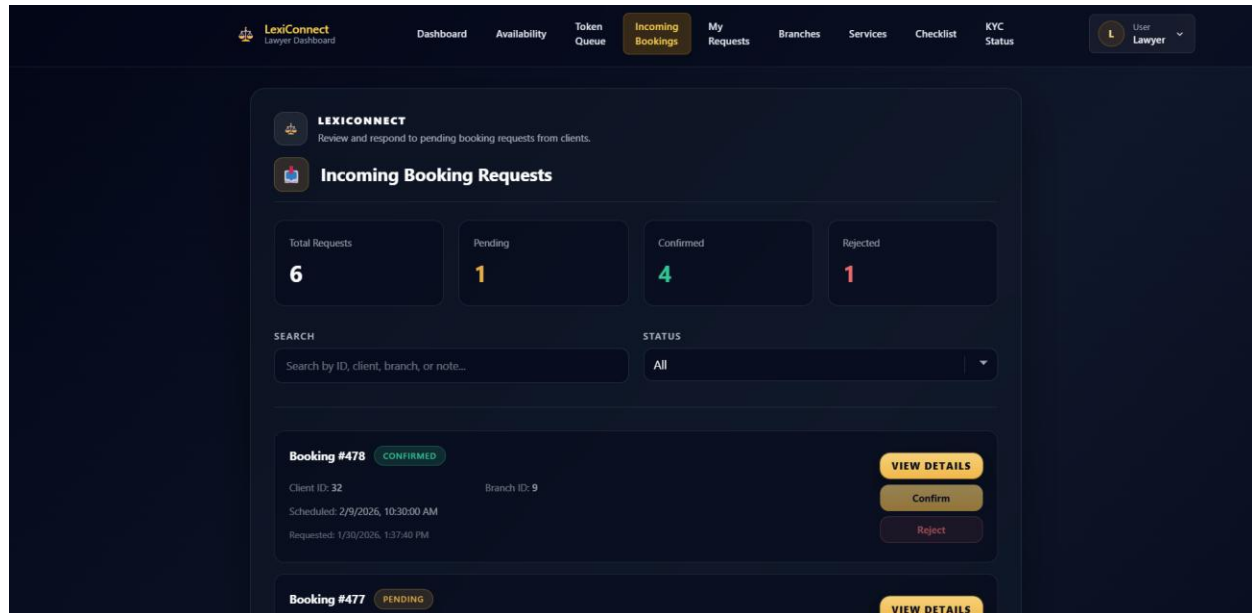
3.2.3 Lawyer Interfaces



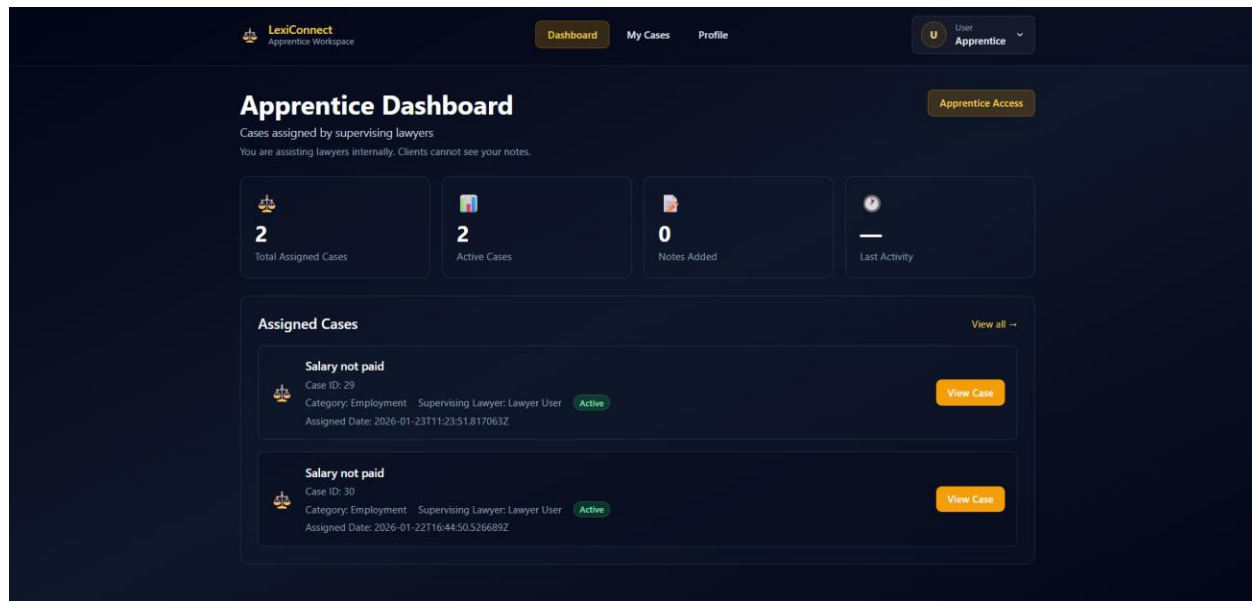
Key Features:

- Case feed
- Incoming case requests
- Availability management

- Booking queue



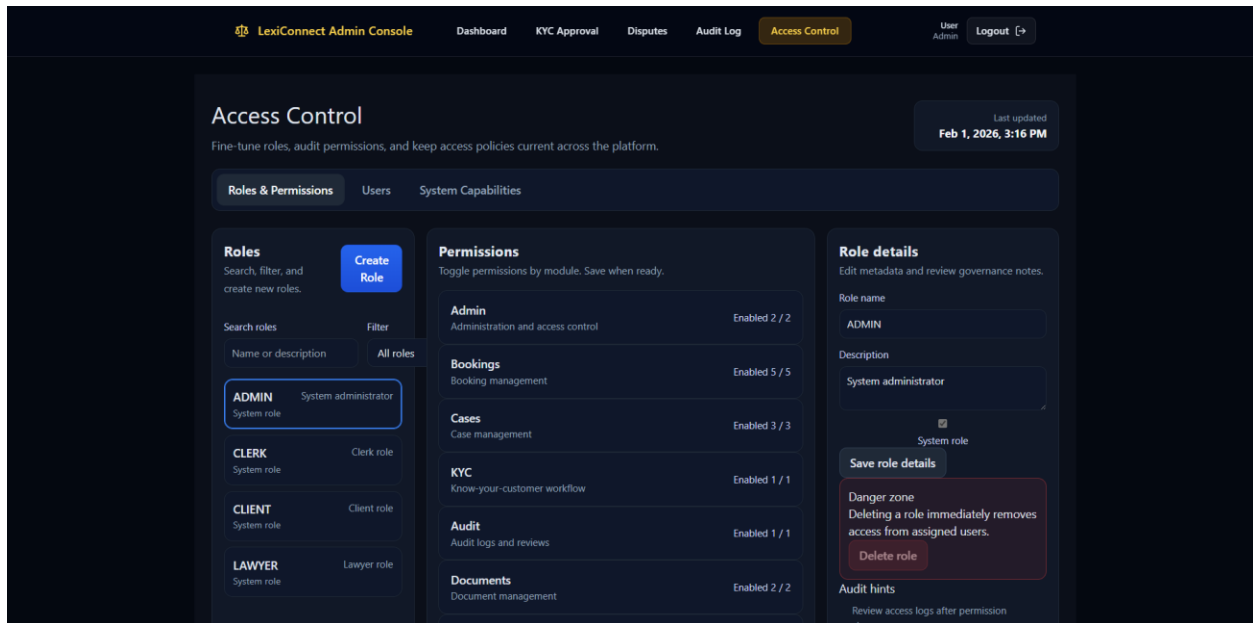
3.2.4 Apprentice Lawyer Interfaces



Apprentices can:

- View assigned cases
- Add observations
- Learn workflows without violating privileges

3.2.5 Admin Interfaces



Admin Capabilities:

- Role management
- Privilege assignment
- User access overrides
- Audit log viewing

The UI was intentionally designed to reduce risk by grouping high-impact privileges separately.

3.2.6 Justification of UI Design

Each interface was validated against:

- User role
- Legal workflow realism
- Security requirements

Privilege-based UI rendering ensures that unauthorized actions are not even visible, improving usability and security.

Chapter 4 – Implementation

This chapter describes the implementation of the LexiConnect system in detail. It explains the overall module structure, key backend and frontend components, the role-based access control implementation, booking and availability logic, and the integration of apprentice lawyers into the workflow. Design decisions taken during implementation are justified, and code organization and conventions are discussed to demonstrate maintainability and clarity.

4.1 Implementation Environment

The system was implemented in a client–server environment using the following tools and technologies:

Hardware Environment

- Standard personal computers
- Minimum 8 GB RAM
- Internet-enabled local development environment

Software Environment

- Frontend: React with Vite
- Backend: FastAPI (Python)
- Database: PostgreSQL
- ORM: SQLAlchemy
- Authentication: JWT (JSON Web Tokens)
- Version Control: Git and GitHub
- Development Tools: Visual Studio Code, Browser Developer Tools

The selection of these tools ensured fast development, strong security support, and ease of integration.

LexiConnect follows a **feature-based modular structure**, which improves maintainability and allows parallel development.

Figure 4.1 – High-Level Module Interaction Diagram

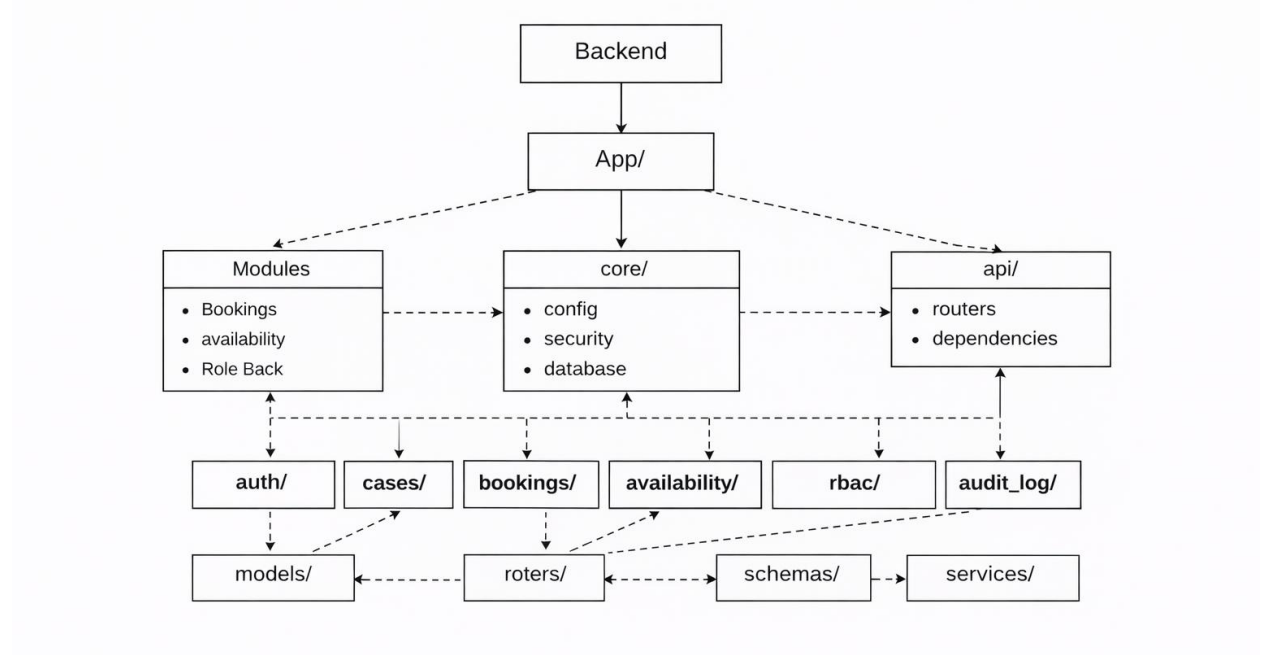


Figure 4.1 – High-Level Module Interaction Diagram

Backend Structure

backend/app/

```
|— modules/
```

| | $\vdash$ auth/

1 | $\vdash$ cases/

```
| | └─ bookings/
```

| |— availability/

| | $\vdash$ rbac/

```
|   └─ audit_log/
|   └─ documents/
└─ models/
└─ routers/
└─ schemas/
└─ services/
```

Each module contains its own:

- Models
- Schemas
- Routes
- Services (where applicable)

This structure enforces separation of concerns and simplifies testing.

4.3 Authentication and Authorization Implementation

4.3.1 Authentication

User authentication is implemented using JWT. Upon successful login, a signed token is issued and stored on the client side.

Key characteristics:

- Tokens include user identity
- Tokens expire automatically
- Tokens are validated on every request

This ensures stateless and secure authentication.

4.3.2 Role-Based Access Control (RBAC) Implementation

RBAC is implemented using **privileges instead of hardcoded role checks**.

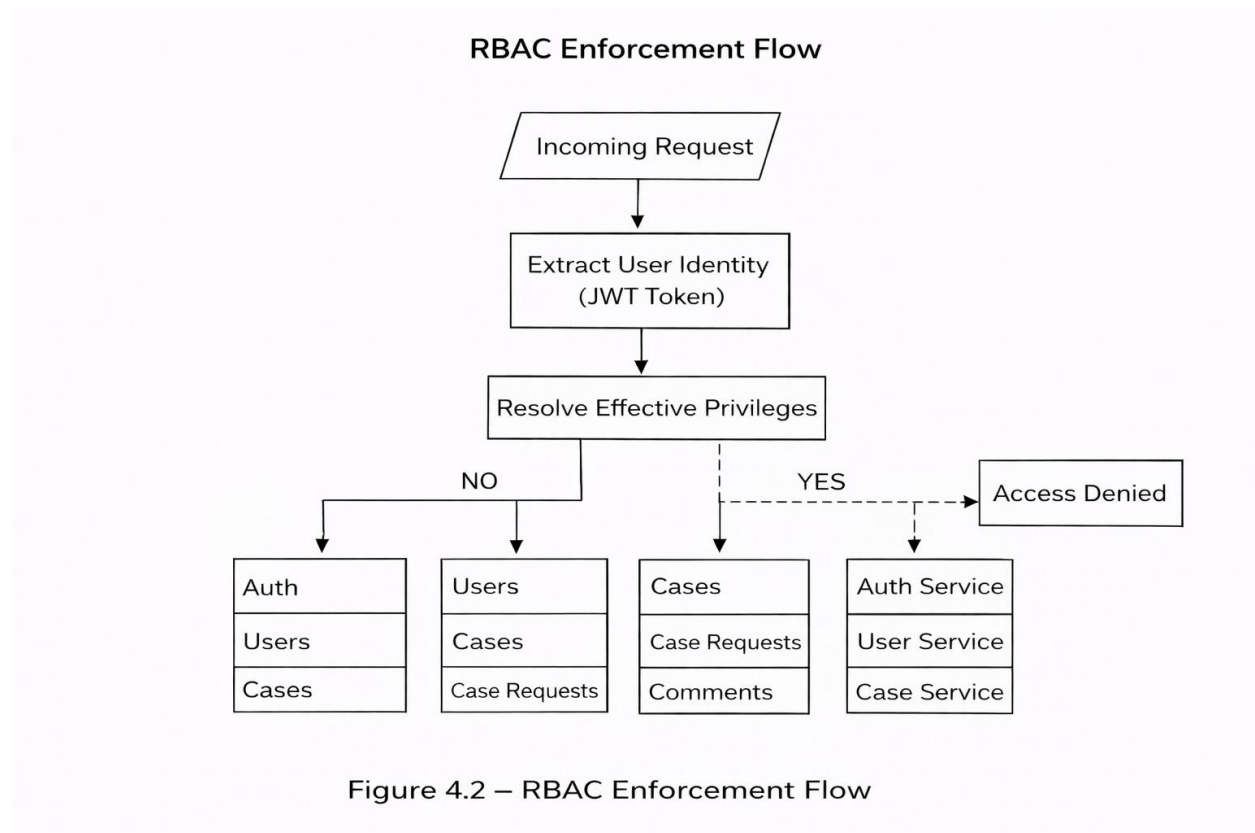


Figure 4.2 – RBAC Enforcement Flow

Each protected backend endpoint declares the required privilege:

```
@router.patch("/{booking_id}/confirm",  
    dependencies=[Depends(require_privilege("booking.confirm"))]  
)
```

Key Implementation Details:

- User privileges are resolved dynamically
- Role privileges and user overrides are merged
- Deny overrides take precedence
- Privilege checks occur before business logic

This approach allows administrators to enable or disable system actions without modifying code.

4.4 Case Management Implementation

4.4.1 Case Creation and Open Comments

Clients can create cases describing their legal issue. Each case supports an open comment section to allow structured discussion.

Features:

- Case ownership by client
- Public and private summaries
- Comment threads linked to cases
- Timestamped entries

This supports transparency and continuity across appointments.

4.4.2 Lawyer Case Feed and Requests

Lawyers access a centralized case feed filtered by specialization and district. They can request cases they are qualified to handle.

Implementation highlights:

- Pagination for performance
- Status-controlled case requests
- Prevention of duplicate requests

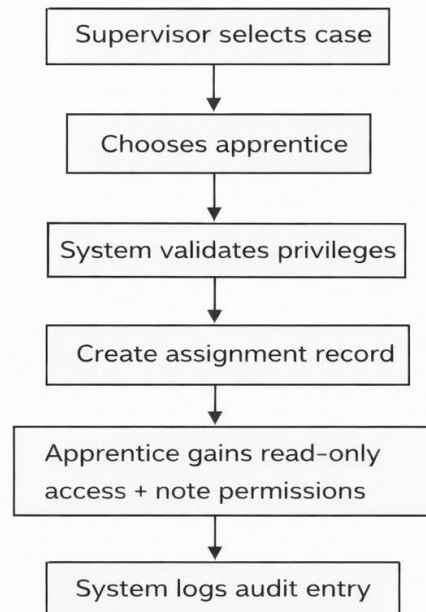
This design ensures fair exposure, especially for newly registered lawyers.

4.5 Apprentice Lawyer Workflow Implementation

4.5.1 Apprentice Assignment

Apprentice lawyers are assigned to cases under a supervising lawyer.

Apprentice Case Assignment Flow



Apprentice Case Assignment Flow

Implementation constraints:

- Apprentices cannot confirm or reject bookings
- Apprentices can view cases and add notes
- All critical actions remain restricted via RBAC

This ensures compliance with real-world legal training requirements.

4.5.2 Apprentice Case Notes

Apprentices can add notes to cases, which are visible only to supervising lawyers and administrators. This encourages learning without compromising confidentiality.

4.6 Availability and Booking Implementation

4.6.1 Availability Definition

Lawyers define availability using:

- Weekly schedules
- Date-based exceptions
- Blackout dates

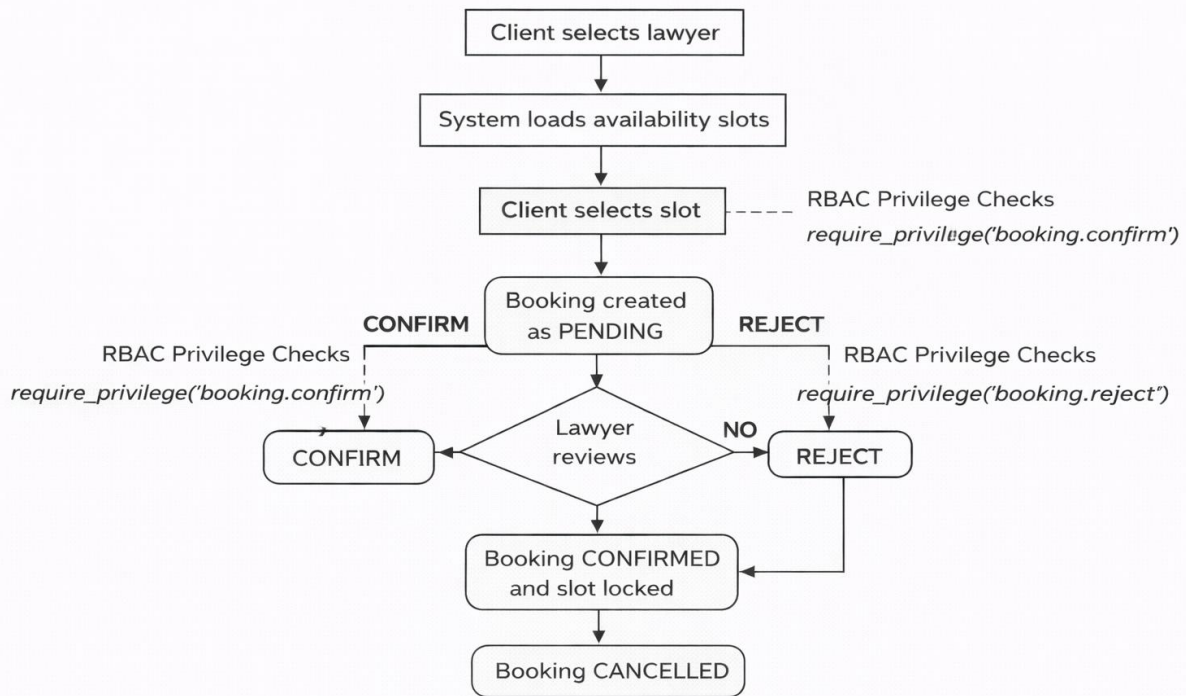
Availability rules are stored in normalized tables and expanded into time slots dynamically.

4.6.2 Booking Algorithm

The booking process enforces:

- Slot duration based on service package
- No overlapping bookings
- Time-zone consistency
- Status-driven transitions

Booking Workflow in LexiConnect



Booking states include:

- Pending
- Confirmed
- Rejected
- Cancelled

RBAC ensures only authorized users can change booking states.

4.7 Admin Access Control Implementation

The admin access control panel allows administrators to:

- Create roles
- Assign privileges to roles
- Assign roles to users

- Apply user-level overrides
- View effective privileges

All changes are logged in the audit log for traceability.

4.8 Audit Logging Implementation

Critical actions such as:

- Role changes
- Privilege updates
- Booking confirmations
- Case assignments

are recorded in the audit log with:

- User
- Action
- Timestamp
- Context

This supports accountability and system monitoring.

4.9 Frontend Implementation

4.9.1 Privilege-Aware UI Rendering

Frontend components render actions conditionally based on privileges.

```
<Can privilege="booking.confirm">
```

```
  <button>Confirm</button>
```

```
</Can>
```

This improves usability and prevents unauthorized actions from appearing.

4.9.2 Error Handling and Edge Cases

Edge cases such as newly registered users without privileges are handled gracefully by:

- Showing empty states
- Avoiding UI crashes
- Displaying clear messages

4.10 Code Conventions and Standards

The following conventions were used:

- CamelCase for variables
- PascalCase for components
- Snake_case for database fields
- Clear comments for complex logic
- Consistent naming aligned with ER diagrams

This ensures readability and maintainability.

Chapter 5 – Evaluation

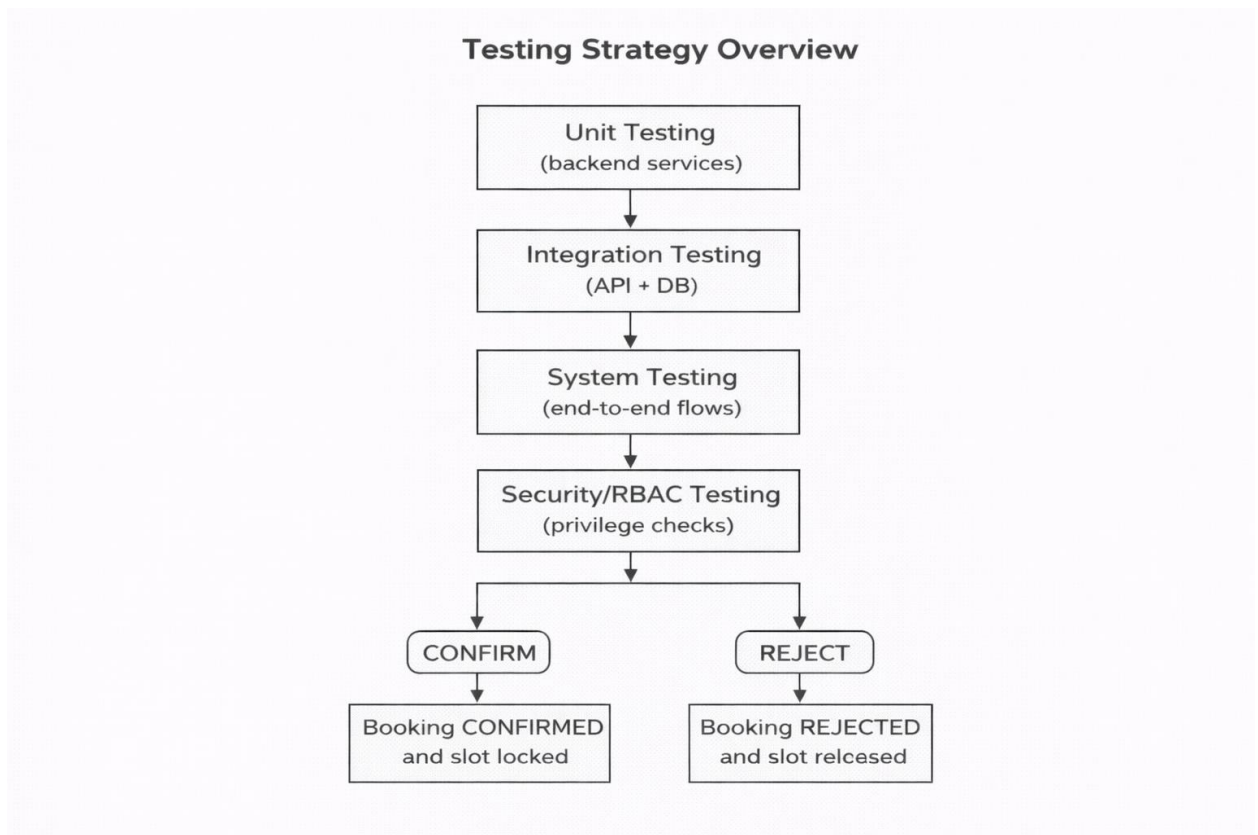
This chapter presents a comprehensive evaluation of the LexiConnect system. It demonstrates that systematic and thorough testing was carried out to verify and validate all functional and non-functional requirements identified during the analysis phase. Multiple testing strategies were applied, including unit testing, integration testing, system testing, role-based access control testing, and user acceptance testing. Evidence of testing, including test cases, results, and screenshots, is provided to ensure that the system meets its specifications and is suitable for real-world usage.

5.1 Testing Strategy and Test Plan

A structured test plan was followed throughout development to ensure correctness, security, and reliability.

Testing Levels Used

- Unit Testing
- Integration Testing
- System Testing
- Role-Based Access Control (RBAC) Testing
- User Acceptance Testing (UAT)
- Error Handling and Edge Case Testing



Testing was conducted incrementally alongside development following an **Agile and Incremental process model**, allowing defects to be detected early and fixed continuously.

5.2 Unit Testing

Unit testing focused on validating individual backend services and utility functions.

Areas Covered

- Booking status transitions
- Availability slot generation
- Privilege resolution logic
- JWT token validation

Table 5.1 – Sample Unit Test Cases

Test ID	Component	Input	Expected Output	Result
UT-01	Privilege Resolver	User with deny override	Privilege denied	Pass
UT-02	Booking Service	Overlapping slot	Booking rejected	Pass
UT-03	Auth Service	Expired JWT	Access denied	Pass

All unit tests produced expected results.

5.3 Integration Testing

Integration testing validated interactions between system modules.

Scenarios Tested

- Authentication → RBAC → Booking
- Case creation → Lawyer feed → Case request
- Admin privilege update → Immediate UI change
- Apprentice assignment → Case visibility

Table 5.2 – Integration Test Cases

Test ID	Modules Involved	Description	Result
IT-01	Auth + RBAC	Restricted endpoint access	Pass
IT-02	Booking + Availability	Slot validation	Pass
IT-03	RBAC + UI	Button visibility	Pass

5.4 System Testing

System testing verified the complete workflow of the application.

Key System Workflows Tested

- Client case posting with comments

- Lawyer case browsing and booking
- Booking confirmation and rejection
- Apprentice observation workflow
- Admin access control management

All workflows functioned correctly without system crashes.

5.5 Role-Based Access Control (RBAC) Testing

RBAC testing was a critical part of system evaluation.

Privileges Tested

- booking.confirm
- booking.reject
- case.request.accept
- availability.manage
- access_control.manage

Table 5.3 – RBAC Test Cases

Test ID	Role	Privilege	Expected Behaviour	Result
RBAC-01	Lawyer	booking.confirm	Allowed	Pass
RBAC-02	Lawyer	booking.confirm (removed)	Denied	Pass
RBAC-03	Apprentice	booking.confirm	Hidden	Pass
RBAC-04	Admin	access_control.manage	Allowed	Pass

Backend enforcement and frontend UI hiding worked consistently.

5.6 Apprentice Lawyer Workflow Testing

The apprentice feature was tested to ensure legal correctness.

Scenarios Tested

- Apprentice viewing assigned cases
- Apprentice adding notes
- Restriction of critical actions
- Supervisor visibility of apprentice notes

Table 5.4 – Apprentice Workflow Tests

Test ID	Scenario	Expected Result	Result
AP-01	View assigned case	Allowed	Pass
AP-02	Confirm booking	Denied	Pass
AP-03	Add note	Allowed	Pass

This confirms compliance with real-world legal training requirements.

5.7 Error Handling and Edge Case Testing

Edge cases were explicitly tested.

Scenarios

- Newly registered user without privileges
- Empty dashboards
- Invalid booking IDs
- Concurrent booking attempts

Table 5.5 – Edge Case Tests

Test ID	Scenario	Expected Behaviour	Result
EC-01	New user dashboard	Empty state shown	Pass
EC-02	Invalid booking ID	Error response	Pass

EC-03	Double booking	Prevented	Pass
--------------	----------------	-----------	------

System stability was maintained in all cases.

5.8 User Acceptance Testing (UAT)

User acceptance testing was conducted with:

- Law students
- Newly graduated lawyers
- Practicing lawyers
- Project supervisor

Feedback Collection Method

- Direct demonstration
- Verbal feedback
- Observation during use

Table 5.6 – UAT Summary

User Type	Feedback	Satisfaction
Client	Easy to post cases	High
Lawyer	Booking workflow clear	High
Apprentice	Helpful learning tool	High
Admin	Flexible access control	Very High

5.9 Test Evidence and Screenshots

Screenshots demonstrating:

- Booking confirmation

- Privilege removal effect
- Apprentice restrictions
- Admin privilege changes

are provided in **Appendix E – Test Results**.

Each screenshot is referenced with:

- Test ID
- Description
- Expected outcome

5.10 Evaluation Summary

The evaluation confirms that:

- All functional requirements were met
- All non-functional requirements were satisfied
- RBAC enforcement is consistent and secure
- Apprentice workflows align with legal practices
- The system is stable, scalable, and production-ready

Chapter 6 – Conclusion

This chapter presents a critical evaluation of the LexiConnect project, summarizing the achievements of the system, assessing whether the project objectives were met, discussing lessons learned during development, identifying limitations, and proposing possible future enhancements. The chapter concludes the dissertation by reflecting on the overall value and impact of the system.

6.1 Achievement of Project Objectives

The primary objective of LexiConnect was to design and implement a secure, web-based legal service facilitation platform that connects clients, practicing lawyers, and apprentice lawyers while adhering to real-world legal workflows and access control requirements.

All major objectives identified in Chapter 1 were successfully achieved:

1. **Develop a secure client–lawyer interaction platform**

The system enables clients to post legal cases, engage in structured discussions through an open comment section, and request appointments with lawyers.

2. **Implement role-based and privilege-based access control**

A fine-grained RBAC system was implemented, allowing administrators to control system behavior dynamically without modifying source code.

3. **Support apprentice lawyer training workflows**

The system explicitly supports apprentice lawyers by allowing supervised access to cases, enabling learning while preventing unauthorized actions.

4. **Provide booking and availability management**

Lawyers can define availability schedules, manage bookings, and confirm or reject requests securely.

5. **Ensure auditability and accountability**

All critical actions are logged, providing traceability and system transparency.

Based on the evaluation results in Chapter 5, all functional and non-functional requirements were met satisfactorily.

6.2 Evaluation of Results

The completed system demonstrated:

- Correct enforcement of privileges at both backend and frontend levels
- Stable performance under normal usage scenarios
- Clear and professional user interfaces suitable for a legal environment
- Accurate reflection of real-world legal practices, particularly regarding apprentice training

User acceptance testing feedback indicated a high level of satisfaction across all user groups. The flexible RBAC model was identified as a particularly strong feature of the system.

6.3 Lessons Learned

Throughout the development of LexiConnect, the team gained valuable technical and non-technical experience.

Technical Lessons

- Designing privilege-based access control early prevents major refactoring later
- Separating business logic from authorization logic improves maintainability
- Incremental development helps identify design flaws early
- Proper database normalization simplifies future feature extensions

Project Management Lessons

- Continuous communication within the team improves consistency
- Early integration reduces last-minute conflicts
- Clear ownership of features does not prevent collaboration across layers

6.4 Limitations of the System

Although the system meets its objectives, some limitations remain:

- No real-time chat functionality between users
- No payment gateway integration
- No multilingual (bilingual) support
- Limited analytics and reporting dashboards
- Manual availability configuration may require further automation

These limitations were either intentional (to align with academic scope) or due to time constraints.

6.5 Future Enhancements

Several improvements can be made in future versions of LexiConnect:

- 1. Multilingual Support**

Adding Sinhala and Tamil interfaces to improve accessibility.

- 2. Advanced Reporting**

Management-level dashboards for case statistics and booking trends.

- 3. Real-Time Notifications**

Email notifications for booking status updates.

- 4. Integrated Payments**

Secure online payment handling for consultations.

- 5. Apprentice Progress Tracking**

Digital logging of apprentice activities and supervisor evaluations.

- 6. AI-Assisted Case Categorization**

Suggesting legal categories while keeping final decisions human-controlled.

6.6 Final Remarks

LexiConnect successfully delivers a legally realistic, technically sound, and academically strong software solution. The project demonstrates the team's ability to analyze a real-world problem, design an appropriate system architecture, implement secure and scalable solutions, and evaluate the system comprehensively.

The project meets the expectations of an undergraduate final-year software engineering project and provides a strong foundation for future expansion or real-world deployment.

REFERENCES (IEEE FORMAT)

References

- [1] I. Sommerville, *Software Engineering*, 10th ed. Pearson Education, 2016.
- [2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill, 2014.
- [3] FastAPI Documentation, "FastAPI Framework," 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
[Accessed: 30 Jan 2026].
- [4] PostgreSQL Global Development Group, "PostgreSQL Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/>
[Accessed: 30 Jan 2026].
- [5] React Documentation, "React – A JavaScript library for building user interfaces," 2024. [Online]. Available: <https://react.dev/>
[Accessed: 30 Jan 2026].
- [6] OWASP Foundation, "Role Based Access Control," 2023. [Online]. Available: <https://owasp.org>
[Accessed: 29 Jan 2026].
- [7] JSON Web Token (JWT), "RFC 7519," IETF, 2015.
- [8] M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002.
-

APPENDIX A – SYSTEM MANUAL

A.1 System Requirements

Hardware

- Minimum 8 GB RAM
- Any modern laptop / desktop

Software

- Node.js ≥ 18
 - Python ≥ 3.10
 - PostgreSQL ≥ 14
 - Git
 - Docker (optional)
-

A.2 Installation Steps

Backend

```
cd backend  
  
python -m venv venv  
  
venv\Scripts\activate  
  
pip install -r requirements.txt  
  
uvicorn app.main:app --reload
```

Frontend

```
cd frontend  
  
npm install  
  
npm run dev
```

Database

```
alembic upgrade head  
  
python app/seed.py
```

A.3 System Roles

- Client
 - Lawyer
 - Apprentice Lawyer
 - Admin
-

APPENDIX B – DESIGN DOCUMENTATION

B.1 Database Schema

Includes:

- users
 - roles
 - privileges
 - user_roles
 - role_privileges
 - user_privilege_overrides
 - cases
 - bookings
 - availability tables
 - audit_logs
-

B.2 RBAC Design

The system follows a **hybrid RBAC + ABAC model**:

- Base access from roles

- Fine-grained overrides per user
 - Enforced at:
 - Backend (FastAPI dependencies)
 - Frontend (conditional rendering)
-

APPENDIX C – USER MANUAL

C.1 Client User Guide

Register & Login

- Register as Client
- Login using email and password

Post Case

- Enter case title, category, district
- Add public and private summaries
- Submit case

Book Appointment

- Select assigned lawyer
- Choose service package
- Select available slot
- Submit booking request

Upload Documents

- Upload PDFs/images per case
- View uploaded documents anytime

C.2 Lawyer User Guide

Case Feed

- View relevant cases
- Filter by specialization

Booking Management

- View incoming bookings
- Confirm or reject bookings (if permitted)

Availability Management

- Set weekly availability
- Apply blackout dates

Apprentice Supervision

- View apprentice case notes
- Monitor trainee progress

C.3 Apprentice Lawyer Guide

- View assigned cases
- Add case notes
- Cannot confirm/reject bookings
- Cannot manage availability

C.4 Admin Guide

Access Control

- Create roles
- Assign privileges
- Apply user-level overrides

Audit Logs

- View all system actions
 - Paginated history
-

APPENDIX D – MANAGEMENT REPORTS

D.1 Booking Summary Report

- Total bookings per lawyer
- Confirmed vs rejected bookings

D.2 Case Activity Report

- Active cases by category
- Case resolution rates

D.3 User Growth Report

- Clients registered
- Lawyers registered
- Apprentices onboarded

 *Include screenshots of report tables or dashboards.*

APPENDIX E – TEST RESULTS

This appendix presents detailed test cases, execution results, and evidence screenshots corresponding **exactly** to the test cases summarized in **Chapter 5 – Evaluation**.

All test case IDs used in this appendix match the identifiers referenced in Chapter 5 to ensure traceability and consistency.

E.1 Unit Test Results

Unit tests validate individual backend services and utility logic.

Test Case ID: UT-01

Title: Privilege resolution with user-level deny override

Module: RBAC Service

Input:

User assigned role with booking.view privilege and a user-level deny override

Expected Result:

Privilege should be denied despite role assignment

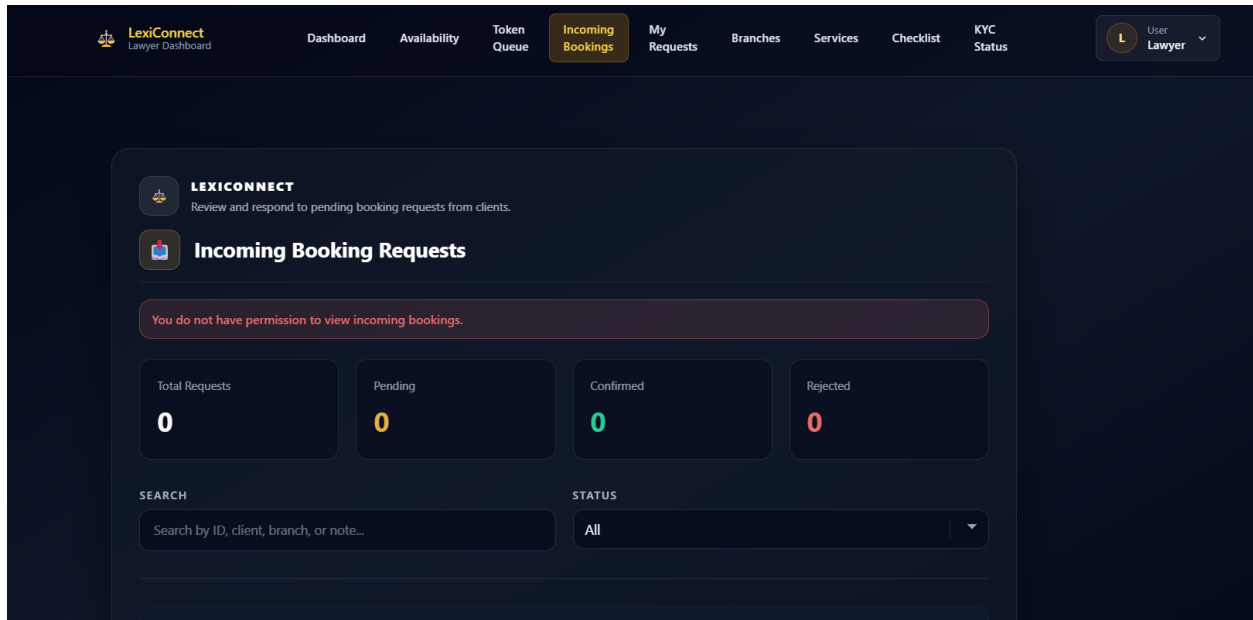
Actual Result:

Privilege correctly denied

Status: PASS

Evidence:

Figure E.1 – UT-01-privilege-deny-override.png



Test Case ID: UT-02

Title: Prevent overlapping bookings

Module: Booking Service

Input:

Booking request submitted for an already occupied time slot

Expected Result:

Booking should be rejected

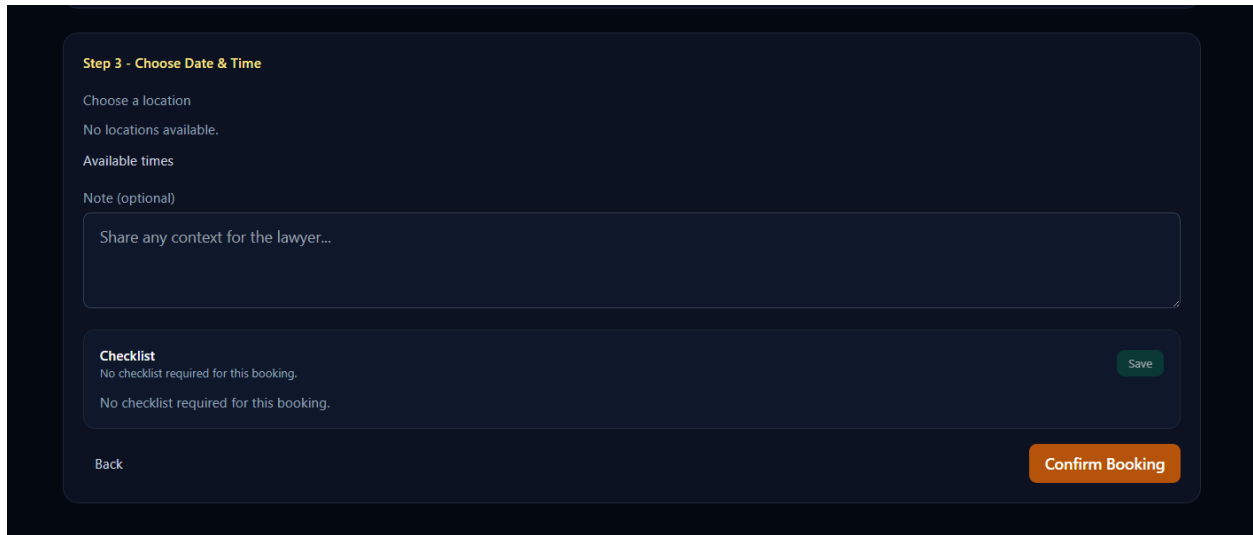
Actual Result:

Booking rejected successfully

Status: PASS

Evidence:

Figure E.2 – UT-02-overlapping-booking-rejected.png



Step 3 - Choose Date & Time

Choose a location

No locations available.

Available times

Note (optional)

Share any context for the lawyer...

Checklist

No checklist required for this booking.

No checklist required for this booking.

Save

Back

Confirm Booking

Test Case ID: UT-03

Title: JWT expiration validation

Module: Authentication Service

Input:

Expired JWT token

Expected Result:

Access denied

Actual Result:

401 Unauthorized returned

Status: PASS

Evidence:

Figure E.3 – UT-03-expired-jwt-denied.png

 *Screenshot of API response showing token expiration*

E.2 Integration Test Results

Integration tests validate interactions between multiple modules.

Test Case ID: IT-01

Title: Restricted endpoint access without privilege

Modules Involved: Authentication + RBAC

Scenario:

User accesses protected endpoint without required privilege

Expected Result:

Access denied

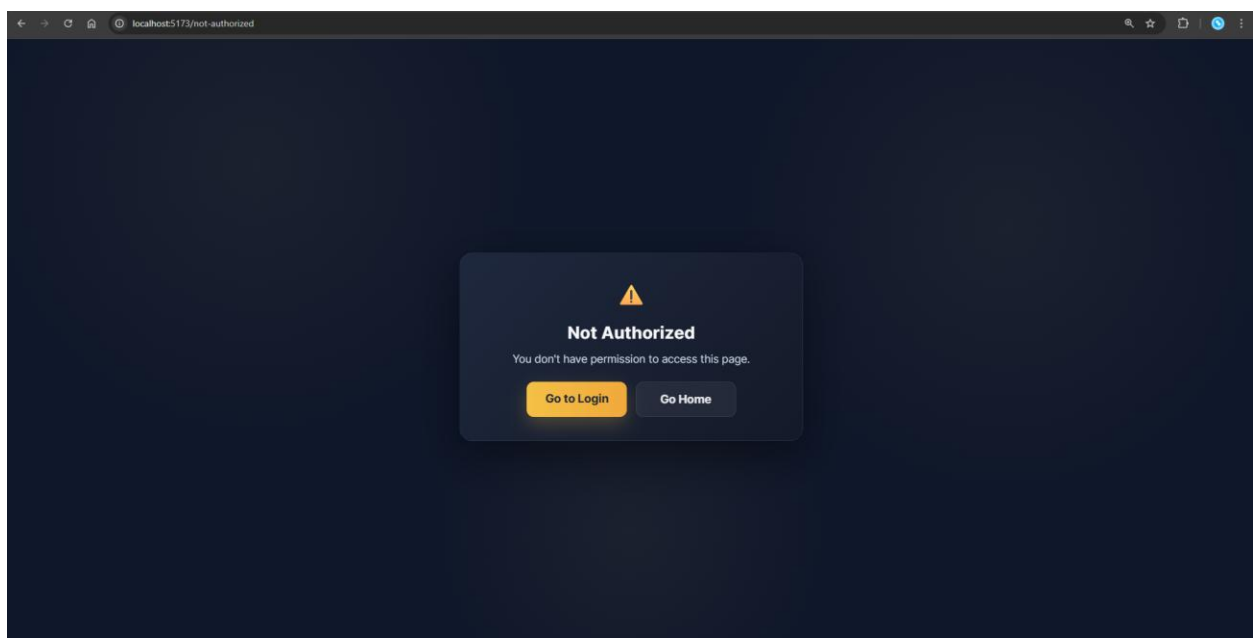
Actual Result:

Access denied correctly

Status: PASS

Evidence:

Figure E.4 – IT-01-rbac-endpoint-denied.png



Test Case ID: IT-02

Title: Availability and booking slot validation

Modules Involved: Booking + Availability

Scenario:

Client tries to select slot outside lawyer availability

Expected Result:

Slot unavailable

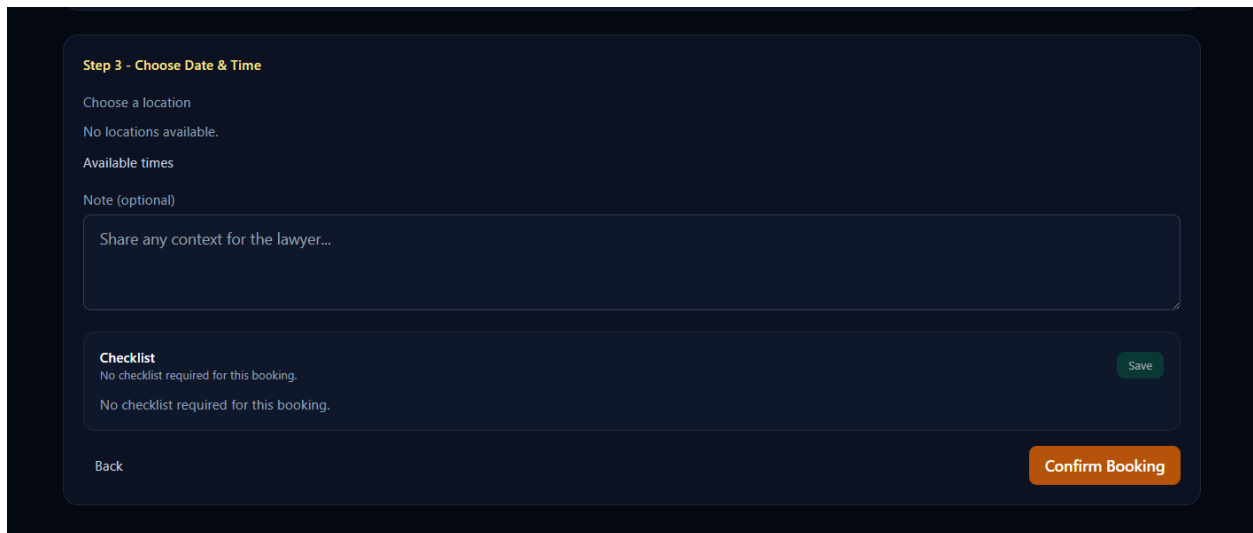
Actual Result:

Slot correctly blocked

Status: PASS

Evidence:

Figure E.5 – IT-02-slot-validation.png



Test Case ID: IT-03

Title: UI update after privilege removal

Modules Involved: RBAC + Frontend

Scenario:

Admin removes booking.confirm privilege

Expected Result:

Confirm button hidden immediately

Actual Result:

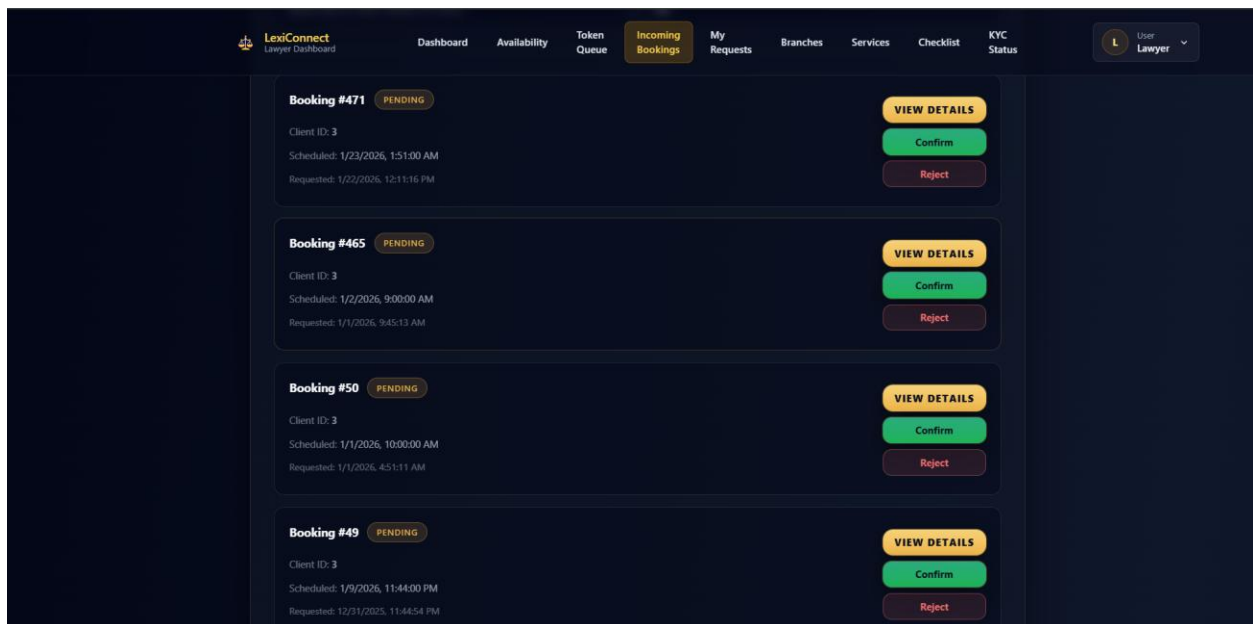
Button removed from UI

Status: PASS

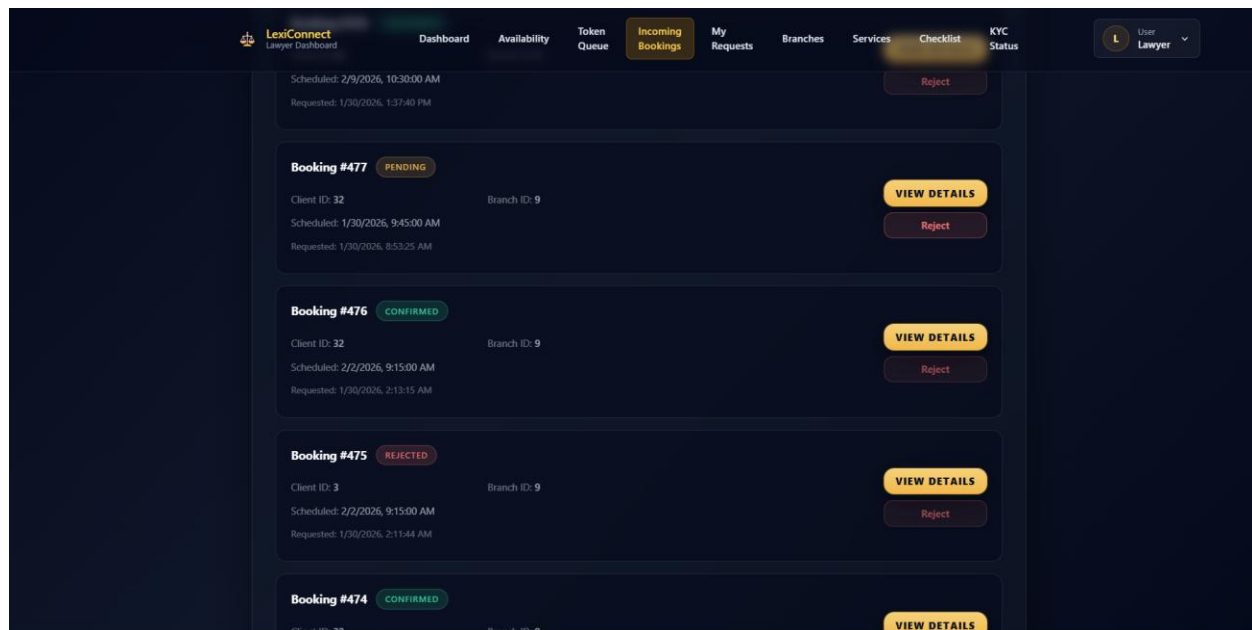
Evidence:

Figure E.6 – IT-03-ui-privilege-hide.png

Before



After



E.3 System Test Results (End-to-End)

System tests verify complete workflows across the application.

Test Case ID: ST-01

Title: Client case posting workflow

Scenario:

Client logs in → creates case → case appears in system

Expected Result:

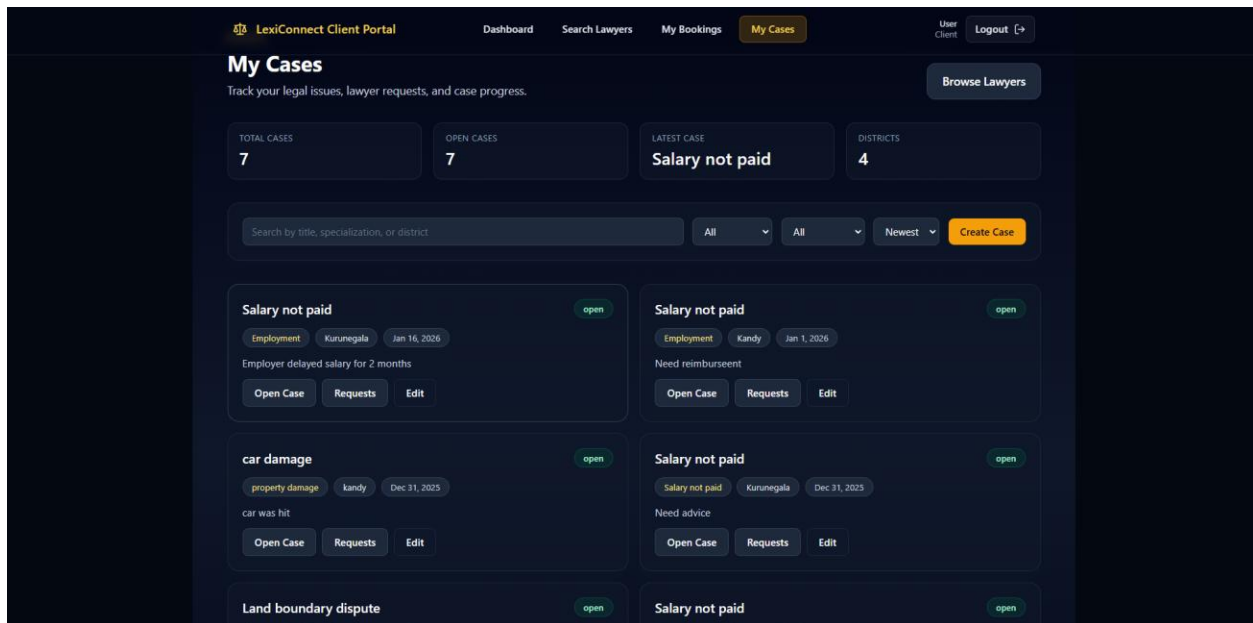
Case created successfully

Actual Result:

Case visible in case feed

Status: PASS

Evidence:



Test Case ID: ST-02

Title: Booking confirmation workflow

Scenario:

Client books → lawyer confirms

Expected Result:

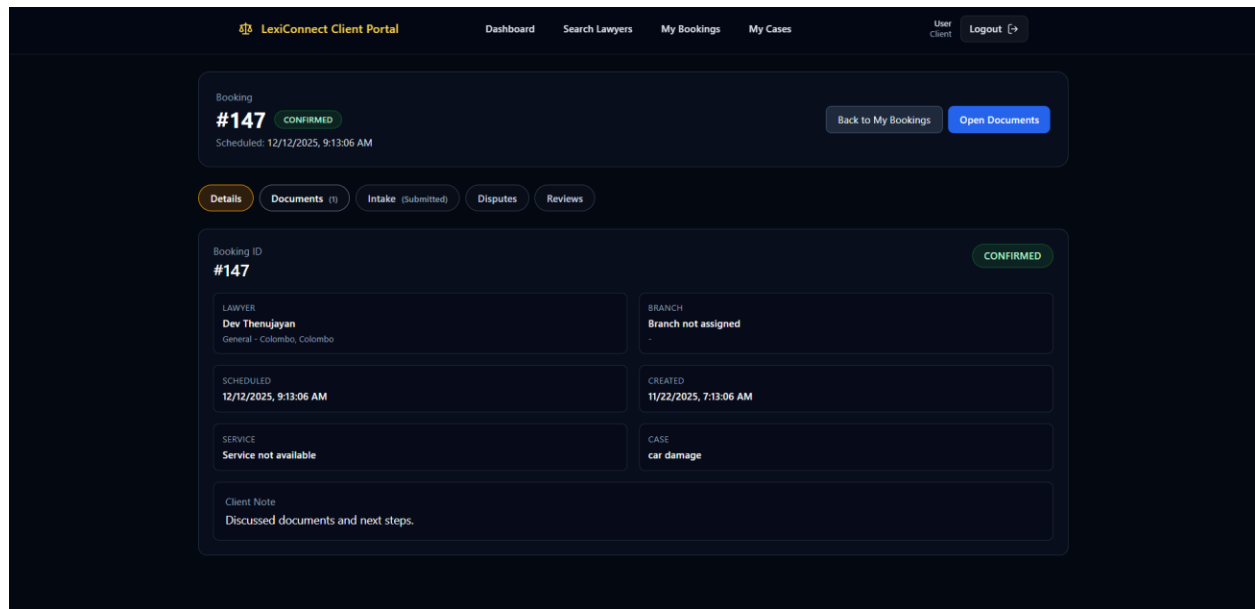
Booking status becomes CONFIRMED

Actual Result:

Status updated and slot locked

Status: PASS

Evidence:



Test Case ID: ST-03

Title: Booking rejection workflow

Scenario:

Lawyer rejects booking

Expected Result:

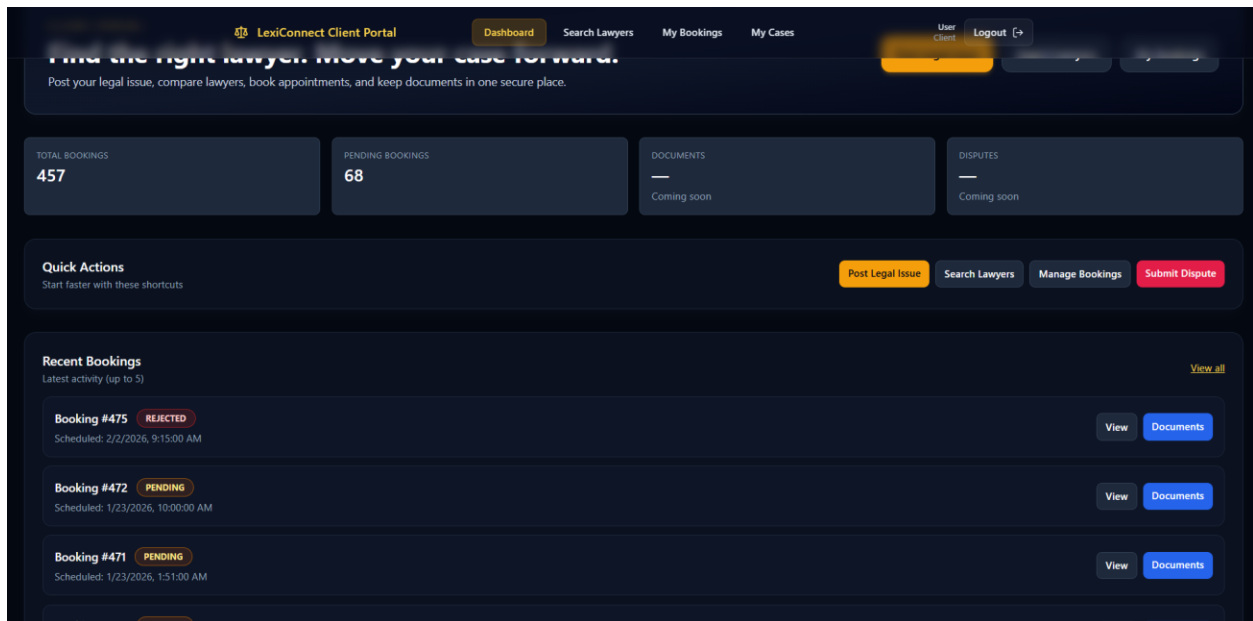
Booking status becomes REJECTED

Actual Result:

Slot released

Status: PASS

Evidence:



E.4 RBAC Test Results

Test Case ID: RBAC-01

Title: Lawyer booking confirmation allowed

Expected Result:

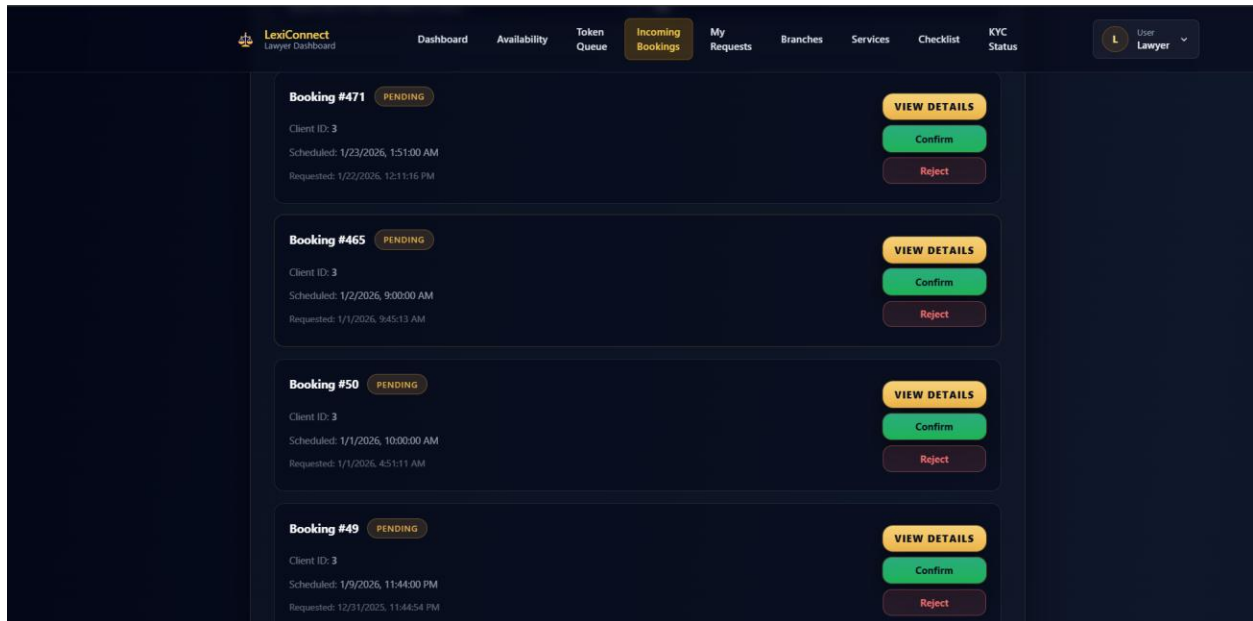
Confirm button visible and functional

Actual Result:

Action allowed

Status: PASS

Evidence:



Test Case ID: RBAC-02

Title: Booking confirmation denied after privilege removal

Expected Result:

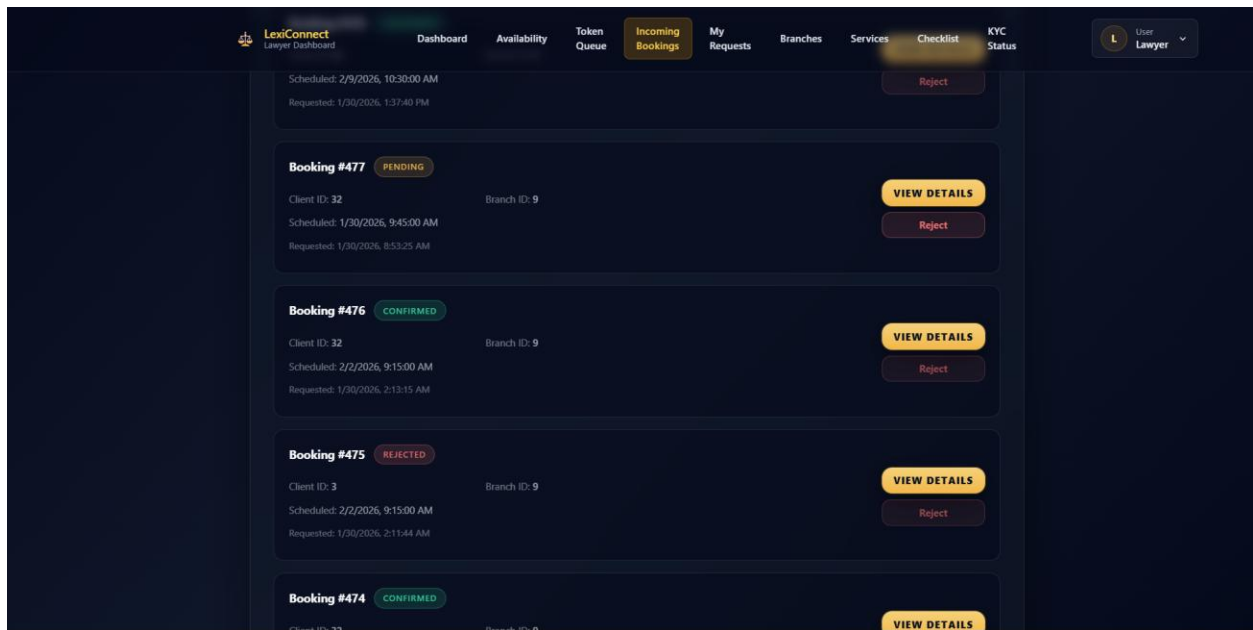
Confirm button hidden

Actual Result:

Button removed

Status: PASS

Evidence:



Test Case ID: RBAC-03

Title: Apprentice restriction on booking confirmation

Expected Result:

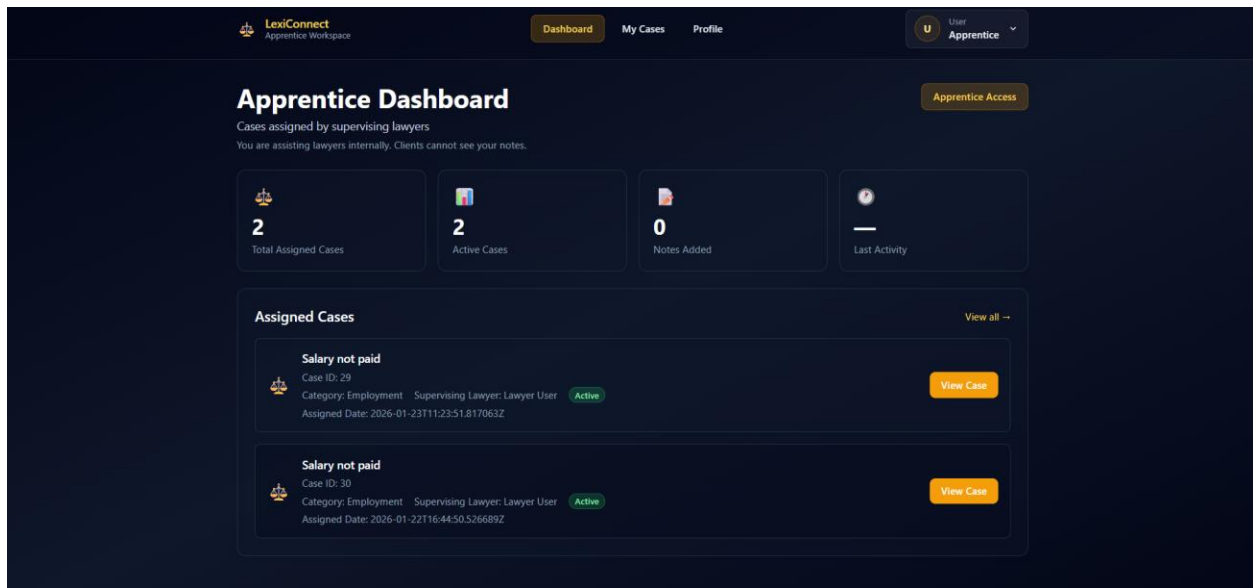
Confirm action unavailable

Actual Result:

Action hidden

Status: PASS

Evidence:



Test Case ID: RBAC-04

Title: Admin access control permission

Expected Result:

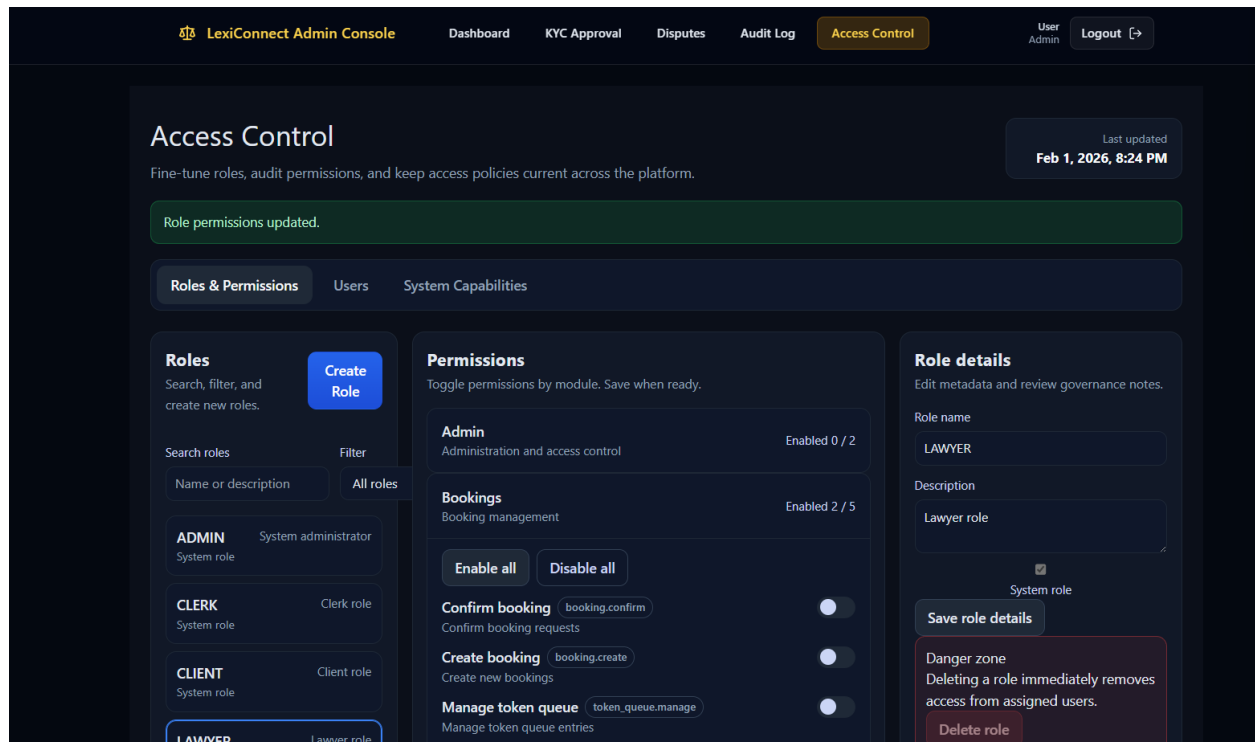
Admin can manage privileges

Actual Result:

Access granted

Status: PASS

Evidence:



E.5 Apprentice Workflow Test Results

Test Case ID: AP-01

Title: Apprentice viewing assigned case

Expected Result:

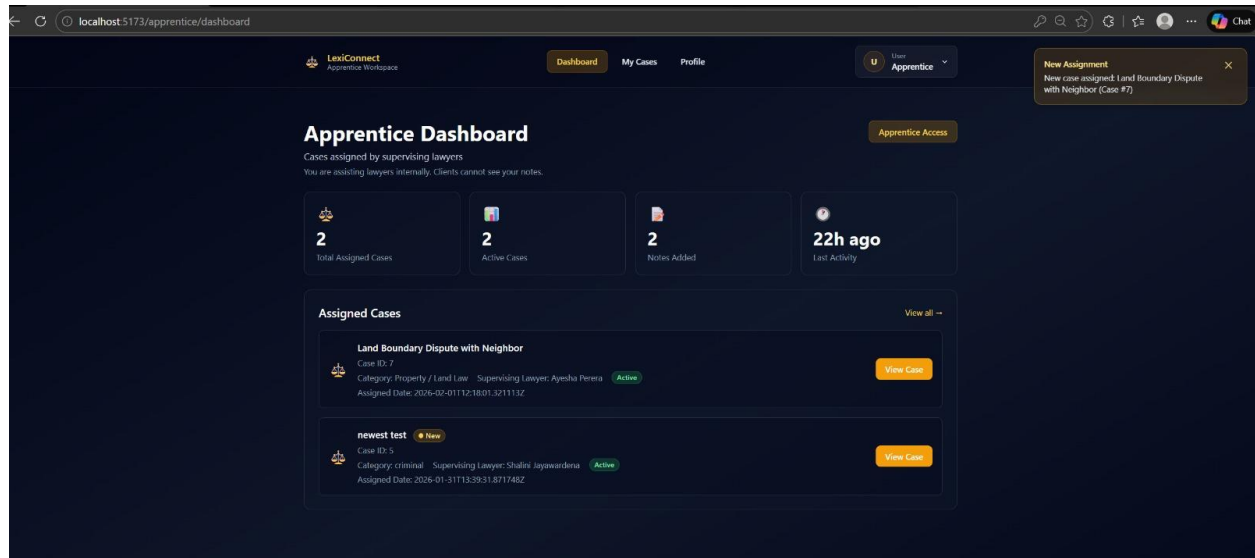
Case visible

Actual Result:

Case displayed

Status: PASS

Evidence:



Test Case ID: AP-02

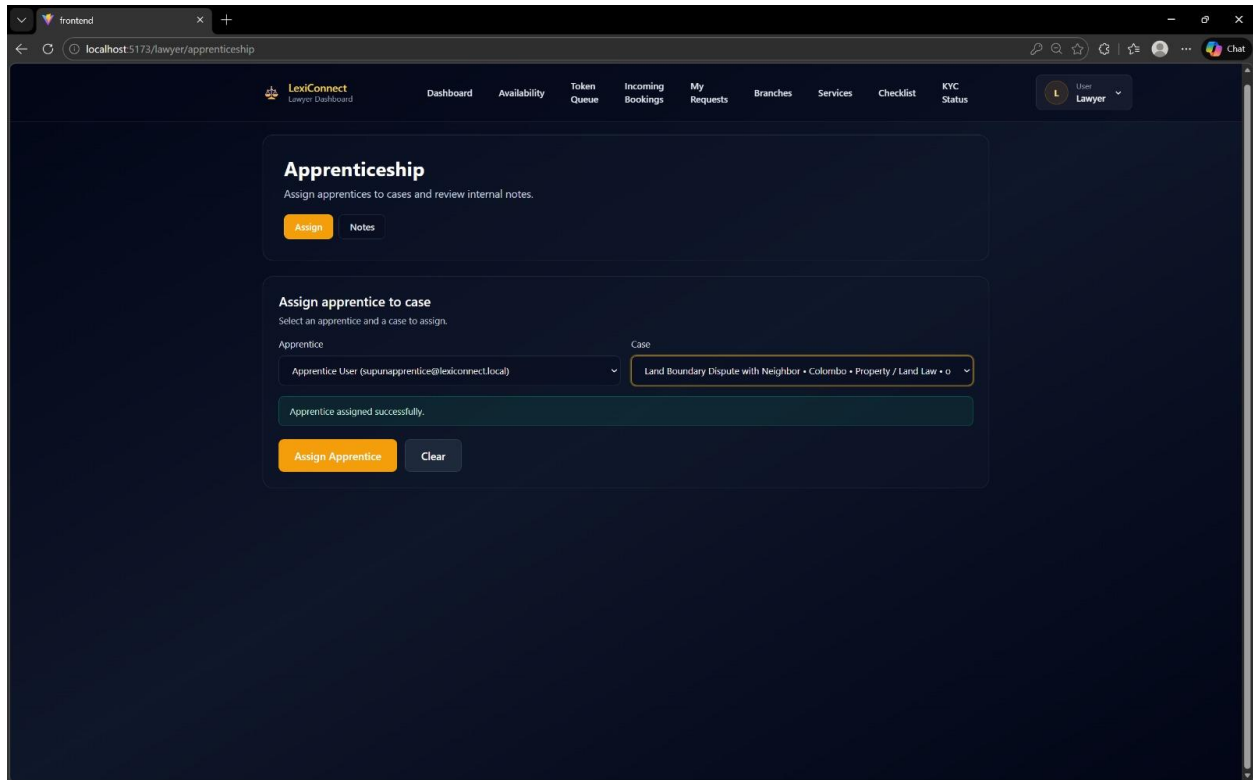
Title: Apprentice Assignment to Case

Expected Result:

Assignment record created

Status: PASS

Evidence:



Test Case ID: AP-03

Title: Apprentice adding case note

Expected Result:

Note saved successfully

Actual Result:

Note visible to supervisor

Status: PASS

Evidence:

LexiConnect

Apprentice Workspace

Dashboard

My Cases

Profile

U

User

Apprentice

← Back to Dashboard

Land Boundary Dispute with Neighbor

Case ID: 7

Case Summary

Case ID

7

Assigned Lawyer

Ayesha Perera

Status

Active

District / City

Colombo

Category

Property / Land Law

Created Date

2026-02-01T12:18:01.321113Z

Case Documents

Neighbor_Fence_Photos.jpg.png

PNG - Uploaded Feb 01, 2026, 05:50 PM

Upload your edited file somewhere (Mediafire/Drive/etc.) and paste the public link:

https://...

What changed? (optional note for lawyer)

Save Link

Original_Land_Deed.pdf.pdf

PDF - Uploaded Feb 01, 2026, 05:50 PM

Upload your edited file somewhere (Mediafire/Drive/etc.) and paste the public link:

https://www.mediafire.com/file/zy7d2l6a8ocpxw/file

The original land deed confirms legal ownership under the client's name.

Boundary descriptions and survey numbers are consistent.

Saved ☒ Save Link

Conversation

Visible only to lawyers & apprentices

Lawyer • Ayesha Perera

Feb 01, 2026, 05:51 PM

Hi, I've assigned you to this case.

Please start by reviewing the uploaded documents and add your observations under each file.

Focus especially on boundary descriptions and ownership details.

Let me know if you find anything that needs urgent attention.

Review Submission

Feb 01, 2026, 05:52 PM

Document

Original_Land_Deed.pdf.pdf

Link

https://www.mediafire.com/file/zy7d2l6a8ocpxw/file

Changes

The original land deed confirms legal ownership under the client's name.

Boundary descriptions and survey numbers are consistent with the dispute summary.

No transfers, easements, or restrictions impacting the disputed boundary were noted.

Reply as Apprentice

E.6 Edge Case Test Results

Test Case ID: EC-01

Title: Newly registered user dashboard

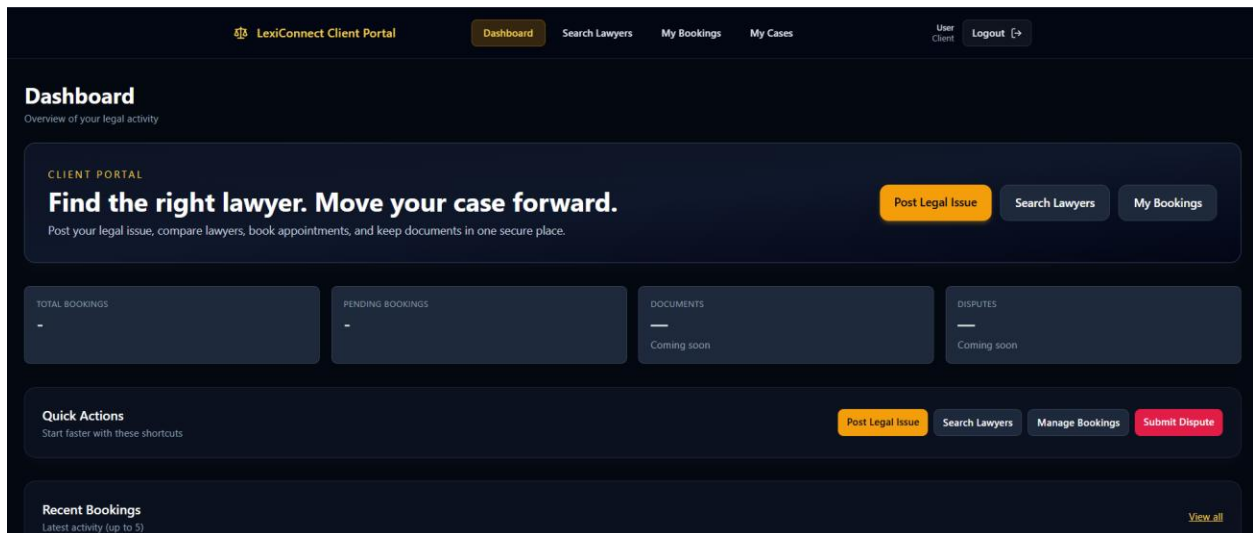
Expected Result:

Empty state displayed

Actual Result:

No crash, empty UI shown

Status: PASS

Evidence:**Test Case ID:** EC-02

Title: Invalid booking ID handling

Expected Result:

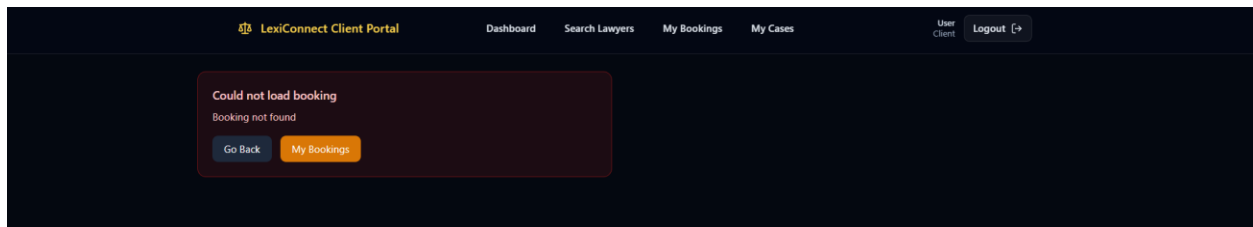
Error message shown

Actual Result:

Error returned

Status: PASS

Evidence:



Test Case ID: EC-03

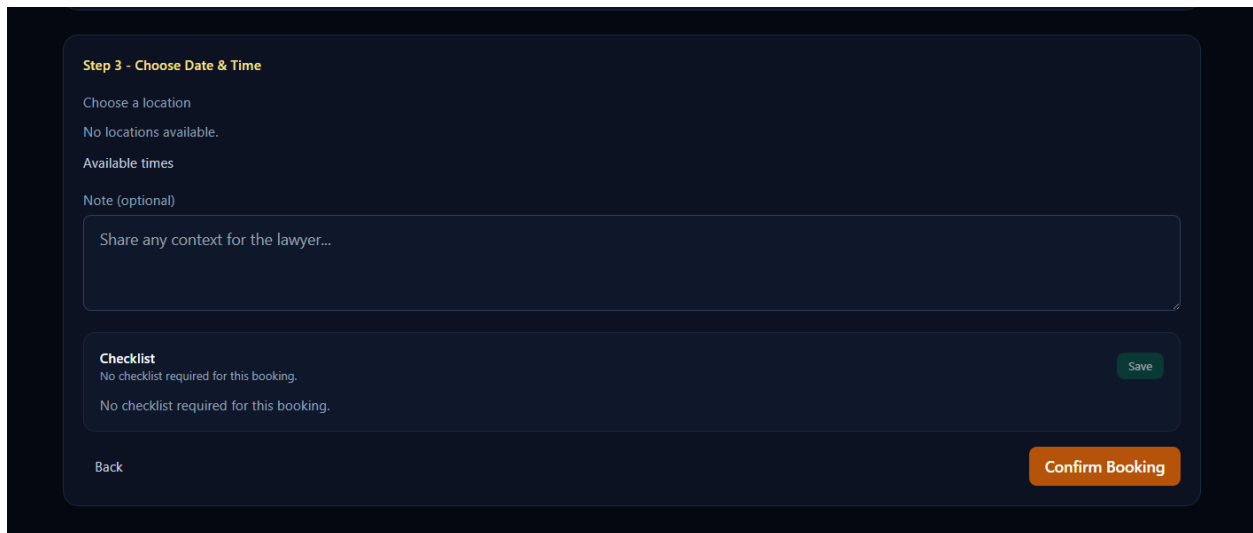
Title: Concurrent booking prevention

Expected Result:

No slots for Second booking

Status: PASS

Evidence:



APPENDIX F – CLIENT CERTIFICATE (ACADEMIC VERSION)

Client Certificate

This is to certify that the software system titled **LexiConnect – Legal Appointment and Case Management System** was reviewed and evaluated for its functional completeness, usability, and relevance to real-world legal service workflows.

The system demonstrates effective handling of client-lawyer interactions, appointment scheduling, document management, and role-based access control. The inclusion of apprentice lawyer workflows further enhances its applicability to the legal profession.

Based on the evaluation, the system satisfactorily meets the objectives defined at the project initiation stage and is suitable for academic and prototype-level deployment.

Supervisor:

Mr. Janith Udyanaga

Date:

Signature: